

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	5
1 АНАЛИТИЧЕСКИЙ РАЗДЕЛ	8
1.1 Анализ существующих решений	8
1.2 Определение критериев сравнения и сравнительный анализ существующих решений	13
1.3 Способы изменения игрового сценария во время игры	18
1.4 Влияние изменения игрового сценария во время игры на пользователя ..	21
1.5 Инструментальные средства разработки	23
Выводы	25
2 СПЕЦИАЛЬНЫЙ РАЗДЕЛ	27
2.1 Разработка концепции и нарративной структуры мира-репозитория.	27
2.2 Формализация основного игрового цикла и пользовательского опыта.....	31
2.3 Проектирование механики переключения веток как инструмента изменения сценария.	35
2.4 Разработка механик взаимодействия: «Code Debugging» и «Anomaly Interaction».....	38
2.5 Проектирование интерфейса (UI/UX) портативного терминала и системы «Data Mining».....	42
Выводы	45
3 ТЕХНОЛОГИЧЕСКИЙ РАЗДЕЛ	47
3.1 Используемые технологии	47
3.2 Информационное обеспечение	50
3.3 Программное обеспечение	54
3.4 Тестирование прототипа, анализ производительности и сценарии дальнейшего масштабирования	73
3.5 Техническое обеспечение	75
Выводы	77
ЗАКЛЮЧЕНИЕ	78
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	81

ВВЕДЕНИЕ

В настоящее время индустрия разработки интерактивного программного обеспечения находится в стадии непрерывной архитектурной и концептуальной эволюции. В связи с насыщением рынка проектами, использующими классические методы построения виртуального пространства, возникает острая потребность в поиске нестандартных паттернов формирования пользовательского опыта. Современные тенденции указывают на высокий спрос на программные продукты, использующие мета-нарративные приемы, а также элементы деконструкции симулируемой реальности. В контексте этого актуальность темы дипломной работы обусловлена необходимостью исследования и алгоритмической реализации интерактивных систем, в которых процессы возникновения программных сбоев, отладки кода и переключения между ветками разработки (системами контроля версий) функционируют не в качестве технических дефектов, а как фундаментальные механики взаимодействия субъекта с окружением. Разработка подобных систем требует создания высоконагруженной архитектуры, способной обрабатывать многослойную топологию уровня без прерывания базового цикла взаимодействия.

Практическая значимость работы заключается в создании полнофункционального вертикального среза (прототипа базового функционала) интерактивного приложения Reverse Assembly, обладающего отказоустойчивой, модульной и масштабируемой программной базой. Спроектированные в ходе разработки архитектурные решения, включающие систему внедрения зависимостей, менеджер конечных автоматов и глобальную событийную шину, могут быть напрямую переиспользованы в качестве универсального технологического фундамента при производстве иных программных продуктов со сложной логикой. Кроме того, реализованные методы диетического (внутриигрового) интерфейса и алгоритмы управления визуальными эффектами

искажений формируют готовый инструментальный аппарат, пригодный для интеграции в коммерческие проекты.

Научная новизна исследования состоит в формализации и программном воплощении парадигмы, при которой эмерджентность и многовариантность процесса достигаются посредством манипуляции слоями реальности и симуляции рутинных задач разработки программного обеспечения. В рамках спроектированной системы впервые синтезированы механики динамической трансформации топологии сцены, устранения коллизий через визуализированную отладку свойств объектов и внедрения показателя программной стабильности кода в качестве первичного ресурса выживания цифрового аватара.

Цель выпускной квалификационной работы заключается в разработке интерактивного игрового решения с возможностью изменения сценария во время игрового процесса.

Для достижения поставленной цели были сформулированы и решены следующие задачи:

1. Провести комплексный анализ предметной области и существующих интерактивных решений, обладающих механикой трансформации виртуального пространства.
2. Разработать концептуальную структуру программного продукта, включая формализацию основного цикла взаимодействия и проектирование проектной документации.
3. Спроектировать децентрализованную архитектуру приложения на основе паттернов внедрения зависимостей, конечного автомата и глобальной событийной маршрутизации.
4. Выполнить алгоритмическую реализацию подсистемы динамического переключения слоев реальности с учетом синхронизации состояния объектов.

5. Запрограммировать контроллер управления субъектом, включающий в себя интеграцию параметра стабильности, модульную боевую систему и логику перемещения.
6. Разработать дидактический графический интерфейс для сканирования программных аномалий и решения логических головоломок в режиме реального времени.
7. Осуществить тестирование разработанного прототипа, провести анализ производительности подсистем и определить пути дальнейшего масштабирования архитектуры.

1 АНАЛИТИЧЕСКИЙ РАЗДЕЛ

В данном разделе будет проведено комплексное исследование теоретических и методологических аспектов построения вариативного игрового процесса. Будет рассмотрена эволюция систем нелинейного повествования, проанализированы существующие интерактивные решения на предмет наличия механик динамического изменения сценария, а также изучен инструментальный аппарат, применяемый для смены состояний виртуального окружения. На основе проведенного анализа будут сформированы функциональные требования к разрабатываемому программному продукту.

1.1 Анализ существующих решений

В рамках данного подраздела проводится всестороннее исследование актуального рынка программных продуктов интерактивной направленности, в которых реализованы механики динамического изменения сценария и концепции многослойности виртуального пространства [1.1, 1.2, 1.3, 1.4]. Изучение существующих решений является фундаментальным этапом проектирования, поскольку оно позволяет выявить наиболее эффективные паттерны взаимодействия, проанализировать технические методы реализации бесшовных переходов и определить концептуальные недостатки современных архитектурных подходов. В качестве объектов исследования были отобраны проекты, демонстрирующие различные парадигмы взаимодействия субъекта с изменяемой средой: Titanfall 2, Dishonored 2, The Stanley Parable и Pony Island.

1.1.1 Игра Titanfall 2

Первым рассматриваемым проектом выступает Titanfall 2, представляющий собой интерактивное решение в жанре шутера от первого лица, разработанное студией Respawn Entertainment. Фундаментальный интерес для данного исследования представляет архитектурная и геймдизайнерская реализация миссии Effect and Cause. В рамках указанного сегмента пользовательскому аппарату предоставляется возможность осуществления мгновенного переключения между двумя диаметрально противоположными состояниями одной локации: разрушенным комплексом в настоящем времени и функционирующим исследовательским центром в прошлом. С технической точки зрения подобный подход требует от вычислительной системы одновременной потоковой обработки и синхронизации двух независимых, но пространственно перекрывающихся наборов данных сцены. Это включает в себя дублирование геометрии коллизий, раздельное вычисление карт освещения, а также независимую обработку скриптов поведения агентов искусственного интеллекта для каждой из временных линий. Изменение сценария преодоления препятствий в данном программном продукте осуществляется не посредством классического триггерного ветвления, а исключительно через физическое перемещение субъекта между пространственно-временными слоями мира [1.5]. Это позволяет обходить непроходимые зоны, использовать альтернативные маршруты и получать тактическое преимущество над противниками. Несмотря на высочайший уровень технического исполнения бесшовного перехода, вариативность сценария в Titanfall 2 остается жестко детерминированной, а концепция изменения мира ограничивается исключительно бинарным переключением без возможности перманентной модификации правил функционирования симулируемой среды. Кроме того, в проекте полностью отсутствуют элементы мета-нарратива, что не позволяет использовать его концепцию в качестве единственного базиса для разрабатываемого решения.

1.1.2 Игра Dishonored 2

Следующим проектом, подлежащим детальному анализу, является Dishonored 2, созданный студией Arkane Studios. В данном программном продукте методологическая парадигма переключения состояний виртуального пространства реализована с более глубокой интеграцией в общую нарративную структуру и логику причинно-следственных связей. Посредством выдачи субъекту специфического внутриигрового инструментального аппарата система транслирует два состояния локации параллельно, используя методы рендеринга через маски трафарета и порталные шейдеры. Главным отличием от ранее рассмотренного решения является предоставление пользователю возможности оказывать перманентное влияние на текущую версию мира путем модификации или уничтожения ключевых элементов в альтернативной временной линии. Архитектура программного продукта выстроена таким образом, что уничтожение объекта в одном состоянии немедленно перестраивает навигационные сетки, систему укрытий, а также алгоритмы патрулирования неигровых персонажей в другом [1.6, 1.7]. Проект иллюстрирует высокую степень интеграции сценарной вариативности непосредственно в базовый игровой цикл. Тем не менее, данная реализация требует колоссальных вычислительных мощностей для поддержания бесшовности, поскольку процессор вынужден обрабатывать физику и логику двух полноценных сцен одновременно. Как и в предыдущем случае, интерактивное решение не эксплуатирует концепцию программных сбоев или взаимодействия с системным кодом в качестве инструмента изменения сценария.

1.1.3 Игра The Stanley Parable

Третьим программным решением для аналитического разбора выступает The Stanley Parable, разработанное студией Galactic Cafe. Фундаментальной основой данного продукта является разрушение традиционных коммуникативных барьеров между пользователем и вычислительной машиной. Вариативность игрового сценария здесь обусловлена не изменением физических параметров трехмерной среды или геометрии коллизий, а системным сопротивлением директивам закадрового голоса, который выполняет функцию алгоритмического распределителя задач. Топология уровней спроектирована с расчетом на предоставление субъекту постоянного выбора между линейным следованием предписанному маршруту и преднамеренным нарушением заложенной логики работы программы. Каждое отклонение от детерминированного пути генерирует уникальную ветвь сценария, отражающую реакцию системы на непредусмотренное поведение пользователя [1.8, 1.9]. Проект демонстрирует высочайший уровень эмерджентности в контексте развития повествования и активно использует мета-нарративные приемы. Однако геометрия сцены и физические свойства объектов остаются преимущественно статичными. Пространство не подвергается динамической трансформации в реальном времени, а смена состояний мира зачастую сопровождается прерыванием базового цикла взаимодействия, что нарушает критерий бесшовности.

1.1.4 Игра Pony Island

Четвертым объектом исследования была выбрана игра Pony Island. Данный программный продукт выступает эталонным примером применения мета-нарратива и интеграции интерфейсов отладки в качестве легитимных элементов

геймдизайна [1.10, 1.11]. Основное взаимодействие субъекта с системой осуществляется через симуляцию графического интерфейса аркадного автомата с преднамеренно нарушенной функциональностью. В процессе активной сессии производится имитация вмешательства в системный код приложения: пользователь вскрывает меню разработчика для получения доступа к переменным, отвечающим за параметры гравитации, скорости перемещения сущностей и положение невидимых триггеров. Изменение сценарной линии и решение пространственных задач достигается путем перепрограммирования самой виртуальной среды. Системные сбои, графические артефакты и ошибки логики трансформируются в ключевые инструменты преодоления нелинейных участков алгоритмического графа. Подобный подход демонстрирует инновационный способ взаимодействия с окружением, полностью нивелируя границу между системным программным меню и виртуальным пространством. Тем не менее, продукт реализован в условиях жестко ограниченной двухмерной проекции, что исключает возможность оценки эффективности примененных методов в контексте сложной трехмерной топологии и многослойного пространства.

Таким образом, подробный разбор указанных интерактивных продуктов обеспечивает формирование комплексного представления о текущем состоянии индустрии в аспекте конструирования вариативных систем. Ни одно из существующих решений не предлагает комбинированного подхода, сочетающего бесшовную трансформацию трехмерной топологии с инструментарием визуальной и алгоритмической отладки кода.

1.2 Определение критериев сравнения и сравнительный анализ существующих решений

ВВ данном подразделе будет произведено формирование системы оценочных критериев, необходимых для осуществления объективного сравнительного анализа рассмотренных ранее программных продуктов. Выделенные параметры базируются на теоретических аспектах построения многослойного виртуального пространства, изученных в рамках предыдущих этапов исследования, и строго пересекаются с фундаментальной целью разрабатываемого интерактивного решения. Для проведения всестороннего сопоставления и выявления концептуальных пробелов в современных подходах к геймдизайну были определены следующие критерии:

- динамическая трансформация топологии локации;
- интеграция мета-нарративных элементов и диегетических интерфейсов;
- степень эмерджентности и вариативности сценария;
- наличие механик симуляции программных сбоев и интерактивной отладки;
- бесшовность пространственно-временных переходов.

1.2.1 Динамическая трансформация топологии локации

Первый критерий определяет способность вычислительной системы осуществлять комплексную реконфигурацию геометрических и коллизионных параметров виртуальной среды в режиме реального времени. Актуальность данного параметра обуславливается исторической эволюцией механик: если на начальных этапах индустрии вариативность реализовывалась преимущественно посредством текстовых узлов, то современные проекты требуют пространственно-временных механик нелинейности. Оцениваемый критерий детерминирует наличие в программном продукте архитектуры, при которой

геометрия и наполнение уровня выступают в качестве основного инструмента формирования вариативного прохождения. Для полного соответствия данному критерию интерактивная система должна поддерживать одновременную обработку независимых наборов данных сцены, включая иерархии коллизий и навигационные сетки. Данный параметр оценивается по бинарной шкале («да» либо «нет»), где положительное значение свидетельствует о возможности манипулирования физической структурой локации для генерации уникальных пространственных головоломок.

1.2.2 Интеграция мета-нарративных элементов и диегетических интерфейсов

Данный критерий отражает наличие в программном продукте концепции мета-игры и механик деконструкции коммуникативного барьера между вычислительной системой и пользователем [1.12, 1.13, 1.14, 1.15]. Интеграция параметра обуславливается необходимостью органичного вплетения системного функционала в структуру вариативного игрового процесса, при котором программное обеспечение открыто транслирует свой статус искусственно спроектированного объекта. В отличие от традиционного подхода, базирующегося на использовании оверлеев поверх игрового пространства, оцениваемый критерий требует трансформации классических элементов управления в диегетические (внутриигровые) компоненты. Таким образом, процессы аналитического сканирования или отладки алгоритмов начинают выступать в качестве основного инструментария для продвижения по ветвям сценарного графа. Оценка по данному параметру осуществляется с использованием бинарной шкалы, подтверждающей вовлечение пользователя в процесс управления сценарием на мета-уровне.

1.2.3 Степень эмерджентности и вариативности сценария

Третий критерий описывает уровень влияния действий пользователя на глобальную или локальную структуру алгоритмического повествования, что обуславливается тенденцией перехода от жестко детерминированных сценарных конструкций к комплексному системному дизайну. Концепция эмерджентности базируется на проектировании атомарных правил, пересечение которых генерирует уникальные, заранее не предусмотренные разработчиком ситуации [1.16, 1.17]. Оцениваемый критерий детерминирует способность интерактивной системы конструировать ситуации, в которых переключение между слоями реальности провоцирует каскадные изменения, инициируя сложную цепь системных взаимодействий. Важным аспектом является обеспечение предсказуемости базовых правил при непредсказуемости итогового результата. Для стандартизированной оценки применяется тернарная шкала («низкая», «средняя» и «высокая»), где высокое значение констатируется при смещении фокуса с ручного конструирования ситуаций на настройку правил системного взаимодействия.

1.2.4 Механики симуляции программных сбоев и интерактивной отладки

Четвертый критерий фиксирует использование графических и алгоритмических аномалий в качестве легитимного инструмента продвижения по сценарному графу. Применение данного параметра продиктовано архитектурной концепцией разрабатываемой системы, базирующейся на принципах систем контроля версий. В проектируемой среде ошибки исполнения кода, коллизии при слиянии данных и системные конфликты должны выступать в качестве визуальных маркеров нестабильных зон, подлежащих

алгоритмическому рефакторингу и отладке со стороны пользователя. Данный параметр оценивается по бинарной шкале («да» либо «нет»).

1.2.5 Бесшовность пространственно-временных переходов

Пятый критерий устанавливает способность аппаратного и программного комплекса осуществлять переключение между слоями реальности без прерывания базового цикла взаимодействия. Необходимость реализации данного критерия обусловлена поддержанием глубокого эффекта погружения, что категорически исключает использование стандартных экранов загрузки при трансформации окружения. Соответствие данному параметру требует внедрения методов асинхронной потоковой подгрузки данных и предиктивного кэширования ресурсов. Для оценки применяется тернарная шкала («отсутствует», «частичная», «полная»).

1.2.6 Сравнительный анализ программных решений

Для проведения стандартизированного сравнительного анализа ко всем сформированным критериям были применены бинарная и тернарная шкалы оценок. Бинарная шкала принимает исключительно два значения, подтверждающие или опровергающие наличие функции, и применяется к критериям динамической трансформации топологии, интеграции мета-нарративных элементов и наличия механик программных сбоев. Тернарная шкала подразумевает три возможных состояния для оценки качественных характеристик. Она применяется для определения степени вариативности сценария, а также для оценки уровня бесшовности перехода. Результаты проведенного сопоставления представлены в таблице 1.1.

Таблица 1.1 — Сравнительный анализ существующих программных решений

Программный продукт	Динамическая трансформация	Интеграция мета-элементов	Вариативность сценария	Механики симуляции сбоев	Бесшовность перехода
Titanfall 2	Да	Нет	Низкая	Нет	Полная
Dishonored 2	Да	Нет	Средняя	Нет	Полная
The Stanley Parable	Нет	Да	Высокая	Нет	Частичная
Pony Island	Да	Да	Средняя	Да	Отсутствует

В ходе аналитической обработки данных, зафиксированных в таблице 1.1, было выявлено, что ни один из существующих на рынке программных продуктов не удовлетворяет заявленным функциональным и концептуальным требованиям в полном объеме. Было установлено, что высокобюджетные проекты, такие как Titanfall 2 и Dishonored 2, обладают совершенным техническим аппаратом для реализации полной бесшовной трансформации геометрии в трехмерном пространстве. Однако данные решения полностью игнорируют метанарративный потенциал, не предоставляя субъекту возможности взаимодействия с нестабильным программным кодом. В свою очередь, независимые проекты, представленные The Stanley Parable и Pony Island, предлагают высочайший уровень вариативности на уровне системной логики и активно используют механики симуляции сбоев. Тем не менее, они характеризуются критическими ограничениями в аспекте динамической перестройки сложной трехмерной топологии и не способны обеспечить полную бесшовность процесса при смене состояний мира.

На основании выявленных концептуальных пробелов была подтверждена целесообразность разработки собственного программного решения, синтезирующего передовые алгоритмы бесшовного изменения состояний трехмерного окружения с механиками интерактивной алгоритмической отладки. Опираясь на результаты сравнения конкурентов, были определены следующие функциональные требования к разрабатываемой интерактивной системе:

1. Реализовать оптимизированную систему мгновенного переключения статуса активности иерархических групп объектов для трансформации топологии сцены в реальном времени.
2. Спроектировать децентрализованную архитектуру логики поведения искусственного интеллекта, способную автономно адаптироваться к текущей активной ветви виртуального пространства.
3. Разработать и интегрировать диалогический графический интерфейс пользователя, функционирующий в мировых координатах и имитирующий терминал для взаимодействия с программным кодом локации.
4. Создать специализированный визуальный и алгоритмический аппарат на базе пользовательских шейдерных программ для генерации пространственно-временных аномалий.
5. Обеспечить высокую эмерджентность прохождения локаций посредством предоставления пользователю инструментария для перманентной модификации физических свойств окружения.
6. Внедрить паттерны управления памятью и асинхронную загрузку ассетов с целью достижения абсолютной бесшовности при переходе между альтернативными версиями сценария.

1.3 Способы изменения игрового сценария во время игры

Проектирование нелинейных интерактивных систем требует глубокого понимания методов, посредством которых вычислительная среда способна адаптировать структуру повествования в ответ на действия пользователя. В современной индустрии программного обеспечения выделяется несколько фундаментальных подходов к изменению игрового сценария непосредственно в процессе активной сессии. Данные методы варьируются от простейших

алгоритмов дискретного выбора до комплексных систем непрерывной симуляции.

Первым и наиболее традиционным подходом является метод дискретного логического ветвления. Данный способ базируется на использовании графов состояний, где переход между узлами (событиями сценария) инициируется явным выбором пользователя в моменты алгоритмических пауз. Классическим примером служат диалоговые деревья или текстовые интерфейсы выбора действий. Изменение сценария в таком формате носит точечный характер: вычислительная система регистрирует команду и активирует заранее прописанный массив данных (текст, анимацию, изменение значения переменной). Несмотря на простоту реализации, данный метод обладает низким уровнем эмерджентности, так как пользователь ограничен жестко детерминированным набором вариантов, а сама трансформация сценария редко затрагивает фундаментальные физические или геометрические свойства симулируемого мира.

Вторым, более технологически сложным подходом выступает метод динамической трансформации пространственной топологии. В отличие от текстового или диалогового выбора, данный способ интегрирует вариативность непосредственно в архитектуру виртуального окружения. Изменение сценария достигается путем алгоритмической подмены, скрытия или генерации новых трехмерных объектов, навигационных сеток и зон коллизий в режиме реального времени. Практическая реализация такого подхода требует использования систем асинхронной загрузки данных или алгоритмов мультиплексирования сцен (как в случае с механикой Branch Hopping), когда в оперативной памяти одновременно содержатся несколько версий одной локации. Пользователь, сталкиваясь с непреодолимым препятствием, переключает состояние пространства, тем самым изменяя маршрут прохождения, состав активных противников и доступные точки взаимодействия. Этот метод переводит развитие сюжета из плоскости абстрактных решений в плоскость физического исследования.

Третьим подходом является метод алгоритмической манипуляции свойствами объектов (системная отладка). В рамках данного способа изменение сценария происходит не за счет глобальной смены декораций, а путем точечного вмешательства пользователя в базовые переменные отдельных сущностей. Предоставление субъекту специализированного инструментария (например, режима диагностического сканирования) позволяет выявлять скрытые параметры объектов, такие как масса, вектор гравитации или логические циклы анимаций. Последующее изменение этих параметров (рефакторинг) инициирует цепную реакцию: так, уменьшение массы объекта позволяет переместить его на нажимную плиту, что, в свою очередь, открывает ранее заблокированный шлюз. Данный метод формирует микро-вариативность сценария, где препятствия преодолеваются посредством понимания и модификации законов функционирования виртуальной среды.

Четвертым способом изменения сценария является интеграция алгоритмов системной эмерджентности. Этот подход базируется на отказе от жестко прописанных скриптов поведения в пользу создания набора атомарных правил для агентов искусственного интеллекта и физических систем. Сценарий формируется динамически в результате пересечения этих правил. Например, внедрение состояния оглушения (паралича) или программного сбоя для противников позволяет субъекту не только устранять угрозу, но и использовать обездвиженные модели в качестве временных укрытий или элементов решения пространственных головоломок. Развитие сюжета при таком подходе становится уникальным для каждой игровой сессии симулируемой реальностью [1.18, 1.19], поскольку оно диктуется не триггерами разработчика, а результатами непрерывного взаимодействия множества автономных подсистем.

1.4 Влияние изменения игрового сценария во время игры на пользователя

Динамическая перестройка правил функционирования виртуальной среды и структуры повествования оказывает комплексное психофизиологическое и когнитивное воздействие на субъекта, взаимодействующего с интерактивной системой. Проектирование алгоритмов изменения сценария требует тщательного анализа того, как подобные трансформации воспринимаются человеческим мозгом, с целью минимизации негативных эффектов и максимизации уровня вовлеченности.

Первостепенным аспектом является влияние динамического сценария на чувство агентности (Agency) пользователя. В системах со статичным линейным повествованием пользователь выступает преимущественно в роли наблюдателя, чьи действия сводятся к продвижению по заранее проложенному маршруту. Внедрение механик пространственно-временных трансформаций или алгоритмической отладки объектов кардинально меняет эту парадигму. Предоставление возможности напрямую изменять законы физики, массу объектов или глобальную конфигурацию локации формирует устойчивое ощущение контроля над симулируемой реальностью [1.20, 1.21, 1.22]. Пользователь осознает, что преодоление препятствий зависит не от поиска единственного заложенного разработчиком ключа, а от его способности анализировать уязвимости системы и применять доступный инструментарий для адаптации мира под свои нужды.

Вторым критически важным фактором выступает управление когнитивной нагрузкой и пространственной ориентацией. Мгновенная подмена геометрии уровня или внезапное изменение логики поведения агентов искусственного интеллекта способны вызвать острую дезориентацию. Человеческий мозг требует времени для построения новой ментальной карты окружения. В связи с этим, процесс изменения сценария должен сопровождаться компенсаторными

механизмами. Одним из таких механизмов является использование архитектурных «якорей» — неизменных объектов базовой геометрии (например, магистральных колонн или полов), которые сохраняют свои координаты и форму при любых трансформациях пространства. Кроме того, процесс переключения состояний должен маскироваться посредством аудиовизуальной обратной связи: применение кратковременных графических всплывающих, эффектов сотрясения камеры и специфических звуковых паттернов позволяет мозгу интерпретировать скачкообразное изменение окружения как закономерный результат инициированного действия, а не как техническую ошибку программы.

Третьим аспектом влияния является поддержание эффекта присутствия («магического круга») при использовании мета-нарративных механик. Взаимодействие с системными переменными или интерфейсами конфигурации традиционно разрушает погружение, напоминая субъекту о том, что он управляет компьютерной программой. Для нивелирования данного эффекта применяется концепция диететического проектирования. Оформление элементов управления сценарными изменениями в виде внутриигровых устройств (например, голографического терминала, закрепленного на предплечье цифрового аватара) позволяет сохранить логическую целостность мира. Таким образом, даже операции по просмотру кода или переключению логических веток воспринимаются как естественные действия персонажа внутри вымышленной вселенной, что многократно усиливает вовлеченность в процесс исследования.

Четвертым фактором выступает психофизиологический отклик на симуляцию программных аномалий. Интеграция визуальных графических искажений (глитчей) и аудиовизуального шума активно применяется для управления эмоциональным фоном пользователя. Прямая зависимость интенсивности искажений (например, смещения вершин моделей или хроматической аберрации) от показателя стабильности персонажа формирует интуитивно понятную, но психологически дискомфортную среду при получении критического урона. Деградация визуального ряда, сопровождающаяся алгоритмической блокировкой базовых возможностей перемещения (запретом на

спринт), генерирует чувство тревоги и уязвимости, стимулируя пользователя к поиску безопасных зон или инициированию процесса смены сценария для выживания.

Таким образом, грамотно спроектированные механики изменения игрового сценария не только расширяют вариативность прохождения, но и формируют глубокий когнитивный и эмоциональный отклик, превращая процесс адаптации к нестабильной виртуальной среде в фундамент уникального пользовательского опыта [1.23, 1.24].

1.5 Инструментальные средства разработки

В данном подразделе будет рассмотрен и обоснован выбор программного обеспечения и языковой среды, необходимых для конструирования разрабатываемого интерактивного решения. На основе сформированных ранее функциональных требований был определен следующий инструментальный аппарат:

- кроссплатформенный игровой движок Unity;
- объектно-ориентированный язык программирования C#.

Выбор среды Unity в качестве фундаментальной платформы для разработки обусловлен архитектурными особенностями данного программного обеспечения. Базовой парадигмой функционирования Unity выступает компонентно-ориентированная архитектура. В рамках этой структурной модели каждый функциональный элемент виртуальной сцены представляется в виде базового контейнера `GameObject`, логическое поведение и физические характеристики которого детерминируются набором прикрепленных к нему модулей `Component`. Данный подход обеспечивает высокую степень декомпозиции и переиспользования программного кода, что является

необходимым условием для проектирования сложных иерархических систем управления состояниями.

Применительно к реализации механик переключения между альтернативными версиями пространства, компонентная модель предоставляет встроенный инструментарий для мгновенной трансформации окружения. Активация и деактивация независимых иерархических групп объектов или модификация параметров рендеринга у конкретных модулей позволяет изменять визуальное и коллизионное представление локации без необходимости инициации ресурсоемкого процесса полной перезагрузки уровня. Кроме того, среда Unity обладает развитым конвейером рендеринга, поддерживающим интеграцию пользовательских шейдерных программ. Возможности платформы по работе с постпроцессными эффектами и материалами обеспечат необходимую технологическую базу для симуляции системных сбоев, визуализации пространственных аномалий и создания эффекта бесшовного перехода между состояниями игрового мира.

В качестве основного языка программирования для реализации логики разрабатываемого продукта был выбран C#. Данный язык со статической типизацией предоставляет мощный инструментарий для проектирования сложных масштабируемых программных архитектур [1.13, 1.14]. Наличие автоматического управления памятью существенно снижает вероятность возникновения критических ошибок на этапе исполнения кода, что выступает первостепенным фактором при разработке систем с динамически изменяемым состоянием окружения, где требуется высокая стабильность вычислительных процессов при постоянном перераспределении ресурсов.

Интеграция языка C# с движком Unity позволяет эффективно реализовать классические паттерны проектирования программной логики. Для централизованного управления переключением между альтернативными версиями мира оптимальным решением выступает использование паттерна Singleton, гарантирующего глобальную точку доступа к функциям инициации смены сценарной ветки. Для организации механизма широковещательного

оповещения объектов о смене состояния сцены применяется паттерн Observer, провоцирующий автономную реконфигурацию зависимых сущностей. Управление поведением отдельных элементов окружения, функции которых радикально отличаются в зависимости от текущей версии локации, обеспечивается посредством паттерна State. Таким образом, выбранный инструментальный аппарат в полной мере коррелирует с техническими требованиями к реализации многослойного интерактивного мира.

Выводы

В ходе работы над первым разделом был проведен комплексный теоретический и методологический анализ аспектов конструирования интерактивных систем с вариативным игровым процессом и динамически изменяемым виртуальным окружением.

Были исследованы существующие программные продукты, обладающие механиками пространственно-временной трансформации и элементами деконструкции коммуникативного барьера между пользователем и вычислительной системой. На основе сформированной системы узкоспециализированных критериев был произведен сравнительный анализ, результаты которого продемонстрировали отсутствие на рынке решений, в полной мере объединяющих технологии бесшовного изменения топологии уровня с мета-нарративным инструментарием симуляции программных сбоев.

Опираясь на выявленные концептуальные недостатки проанализированных проектов, были сформированы точные функциональные требования к разрабатываемому программному обеспечению. Сформированный перечень задач послужит фундаментальной теоретической базой для последующего проектирования архитектуры интерактивной системы.

Был определен и детально обоснован инструментальный аппарат, требуемый для алгоритмической реализации заявленного функционала. В качестве оптимальной среды разработки был выбран кроссплатформенный игровой движок Unity в интеграции с объектно-ориентированным языком программирования C#. Также был утвержден перечень применимых архитектурных паттернов проектирования, необходимых для обеспечения модульности, масштабируемости и высокой отказоустойчивости программного кода разрабатываемого продукта.

2 СПЕЦИАЛЬНЫЙ РАЗДЕЛ

В данном разделе будет рассмотрен комплексный процесс проектирования интерактивного приложения, базирующегося на мета-нарративном подходе. Будет представлено формирование концепции виртуального пространства, структурированного по принципам систем контроля версий. Будет описана формализация основного игрового цикла и пользовательского опыта. Особое внимание будет уделено разработке ключевых механик, обеспечивающих динамическое изменение сценария в режиме реального времени, включая алгоритмы переключения слоев реальности и интерактивной отладки программного кода. Кроме того, будет приведено описание процесса проектирования диалектического графического интерфейса пользователя, необходимого для навигации и взаимодействия с симулируемой средой.

2.1 Разработка концепции и нарративной структуры мира-репозитория.

Проектирование интерактивного приложения с возможностью динамического изменения сценария требует формирования детализированной и логически обоснованной базы. В рамках данной работы разрабатывается проект Reverse Assembly. Основным подходом игры выступает мета-нарратив, при котором виртуальное окружение представляет собой не классический вымышленный мир, а заброшенную цифровую среду разработки, структурированную по принципам систем контроля версий. Подобная структура обосновывает возможность управления пространством через техническое взаимодействие с программным кодом.

Сюжетная линия разворачивается в 2034 году вокруг последствий технологического эксперимента корпорации Neuronet Studios. Согласно

разработанной предыстории, в 2027 году корпорация инициировала создание проекта Project Assembly, предполагающего использование нейроинтерфейсов для конструирования виртуальной реальности изнутри. Однако внедрение технологии прямого нейронного взаимодействия привело к критической дестабилизации программного кода и непредсказуемым последствиям для команды инженеров, что послужило причиной экстренной консервации проекта. Главный герой позиционируется как независимый исследователь цифровых данных (датамайнер), который получает удаленный доступ к изолированному серверу и погружается во фрагментированный мир с целью реконструкции событий прошлого и выяснения истинных причин закрытия студии.

В качестве основной идеи, определяющей повествовательную структуру и игровой процесс, используется концепция системы управления версиями [2.1, 2.2]. Виртуальное пространство разделено на независимые слои, каждый из которых отражает определенный этап разработки или степень повреждения данных. Для оптимизации процесса создания прототипа и фокусировки на механике изменения сценария были выделены две ключевые реальности: ветка Main и ветка Alpha.

Слой реальности Main представляет собой основную, наиболее стабильную версию игрового мира. Она характеризуется завершенной геометрией и отсутствием визуальных ошибок, однако содержит скрытые логические узлы и искусственные тупики (заблокированные двери, разрушенные переправы), препятствующие линейному прохождению. В этой ветке располагаются безопасные зоны, где игрок может осуществлять анализ окружения без риска получения урона от противников.

Слой реальности Alpha представляет собой тестовый или поврежденный сектор репозитория. Данная ветка обладает измененной топологией, которая позволяет обойти препятствия, непреодолимые в базовой реальности. При этом в слое Alpha активно генерируются программные аномалии и присутствуют агрессивные агенты искусственного интеллекта, что делает нахождение в данной реальности опасным для пользователя.

Для обеспечения глубокого погружения в игровой процесс и интуитивного понимания контекста среды, архитектурное и визуальное представление выделенных слоев проектируется на основе принципа радикального контраста. Ветка Main характеризуется строгой геометрией, преобладанием холодной цветовой палитры и наличием статического освещения. Данный подход формирует ощущение стерильной, но незаконченной лабораторной среды, где отсутствие динамических теней подчеркивает статус замороженной разработки.

В противовес этому, ветка Alpha визуализируется через применение алгоритмов процедурного шума и программного искажения геометрии. Архитектурные объекты в данной реальности подвергаются фрагментации, а освещение переводится в динамический режим с использованием пульсирующих источников красного спектра. Это на визуальном уровне сигнализирует пользователю о высокой степени опасности и нестабильности системы. Подобное визуальное кодирование исключает необходимость использования дополнительных текстовых подсказок, позволяя мгновенно идентифицировать текущее состояние виртуального мира.

Структурная схема архитектуры многослойного виртуального пространства представлена на рисунке 2.1. Данная схема отражает статические связи между ключевыми компонентами системы.

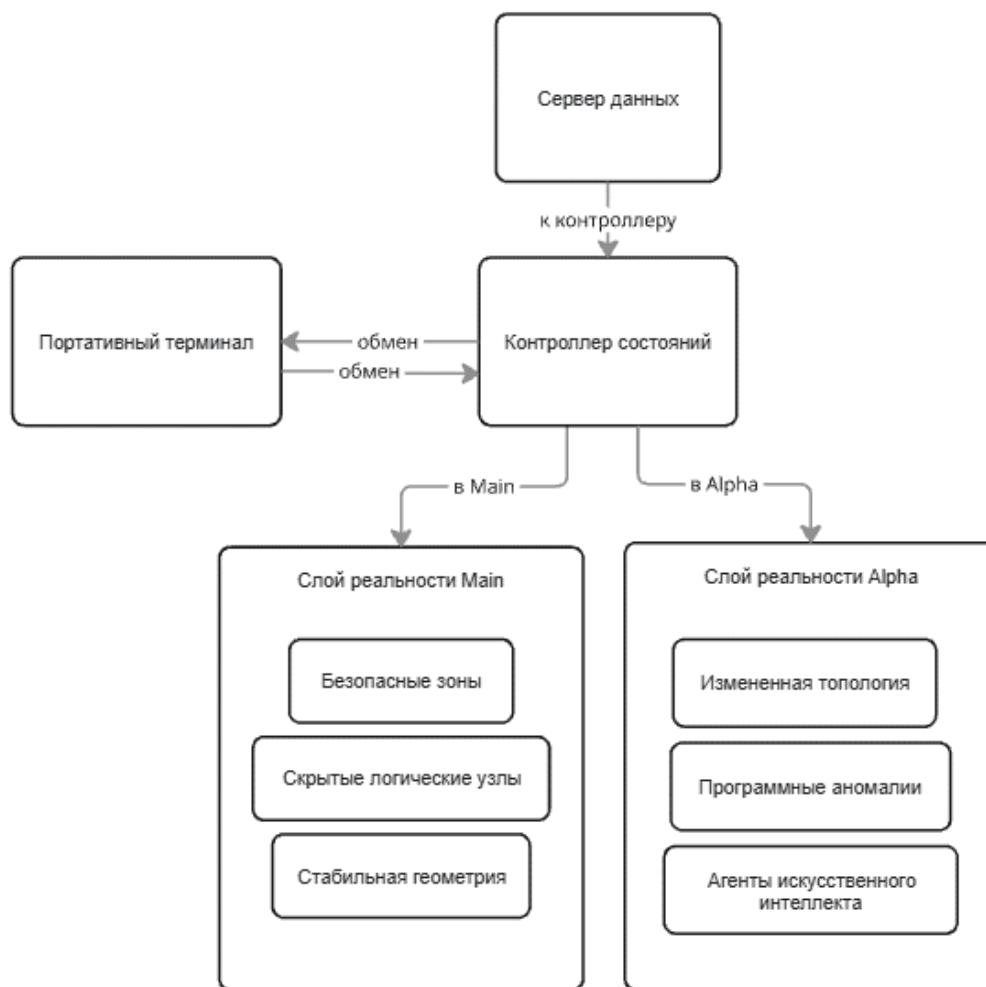


Рисунок 2.1 - Структурная схема архитектуры многослойного виртуального пространства

В центральной части архитектуры располагается контроллер состояний, который управляет глобальным распределением данных. К данному контроллеру поступает информация от сервера данных, содержащего исходные массивы репозитория. Взаимодействие пользователя с системой осуществляется через портативный терминал, который обменивается командами с контроллером состояний. На основе полученных директив контроллер активирует один из двух независимых массивов: слой реальности Main или слой реальности Alpha. Как показано на схеме, слой Main включает в себя безопасные зоны, скрытые логические узлы и стабильную геометрию. В свою очередь, слой Alpha содержит измененную топологию, программные аномалии и агентов искусственного интеллекта.

Такое разделение пространства формирует основу для вариативного прохождения. Пользователь не является пассивным наблюдателем, а выступает в роли активного оператора, способного переключаться между стабильностью и опасностью для продвижения по уровню. Инструментом взаимодействия служит портативный терминал, функционирующий как внутриигровой интерфейс. Данный аппарат позволяет осуществлять сканирование окружения, выявлять логические ошибки в объектах и инициировать процесс перехода между слоями репозитория.

2.2 Формализация основного игрового цикла и пользовательского опыта.

Проектирование интерактивной системы с возможностью изменения сценария во время игры требует строгой формализации базовой повторяющейся структуры действий пользователя. В контексте разрабатываемого проекта Reverse Assembly данный цикл определяет не только механику перемещения персонажа в виртуальном пространстве, но и алгоритм его взаимодействия с многослойной структурой игрового мира. Опыт пользователя выстраивается вокруг непрерывного анализа окружения и принятия тактических решений о целесообразности перехода в альтернативную реальность для преодоления логических или физических преград.

Для обеспечения синхронизации между действиями игрока и вычислительными процессами программного комплекса, основной игровой цикл был разделен на четыре последовательные фазы. Данное разделение фокусируется непосредственно на трансформации пользовательского опыта и развитии сценария:

1. Пространственное исследование и анализ текущего сценария. На начальном этапе пользователю предоставляется возможность свободного

перемещения в пределах активной ветки (например, Main). Игрок осуществляет визуальный сбор информации, изучает геометрию уровня и выявляет элементы, блокирующие дальнейшее продвижение по сюжету. В данной фазе система работает в штатном режиме, обеспечивая непрерывность эффекта погружения.

2. Выявление точек ветвления и работа с терминалом. При обнаружении логического тупика или интерактивного объекта пользователь инициирует процесс сканирования посредством портативного устройства. Алгоритмы системы фиксируют направление взгляда и, при подтверждении контакта с активной зоной, включают внутриигровой пользовательский интерфейс. В этот момент фоновое течение игрового времени может замедляться, акцентируя внимание на необходимости принятия решения.
3. Регистрация выбора и инициация трансформации пространства. После взаимодействия с элементами интерфейса система регистрирует команду на переключение активной ветки. С точки зрения пользователя данный процесс сопровождается интенсивным аудиовизуальным откликом. Применяются эффекты постобработки (симуляция программных сбоев, искажение изображения), которые маскируют ресурсоемкий процесс выгрузки текущих и загрузки новых структурных узлов уровня. Игрок визуально воспринимает данный этап как процесс перезаписи окружающей реальности.
4. Стабилизация нового сценария и продолжение прохождения. После завершения алгоритмических операций по смене состояний объектов и пересчету маршрутов перемещения, визуальные артефакты исчезают. Пользователь оказывается в обновленной конфигурации уровня (например, в ветке Alpha), где ранее заблокированные проходы могут оказаться открытыми, а безопасные зоны — заполненными противниками. Цикл замыкается, и игрок возвращается к первой фазе, но уже в рамках измененного сценарного контекста.

Критически важным аспектом проектирования основного цикла выступает балансировка частоты использования механики трансформации пространства. Неограниченный доступ к переключению веток способен снизить тактическую сложность преодоления препятствий и привести к нарушению эффекта погружения вследствие чрезмерного мелькания визуальных эффектов. В связи с этим в алгоритм основного игрового цикла интегрируется система управления вычислительными ресурсами терминала.

Каждая операция по смене состояния локации расходует специфический внутриигровой ресурс — энергию синхронизации. Восполнение данного ресурса осуществляется в процессе первой фазы цикла посредством нахождения и сбора рассеянных фрагментов данных (блоков кода). Следовательно, пользователь лишается возможности непрерывного и бессмысленного применения механики и вынужден применять аналитический подход, тщательно рассчитывая оптимальный момент для перехода. Введение подобного ограничительного механизма формирует циклы напряжения и разрядки, что является фундаментальным требованием к дизайну интерактивных систем [2.3].

Схема алгоритма функционирования основного игрового цикла представлена на рисунке 2.2. Данная схема описывает последовательность шагов, выполняемых программой и пользователем в процессе прохождения уровня.

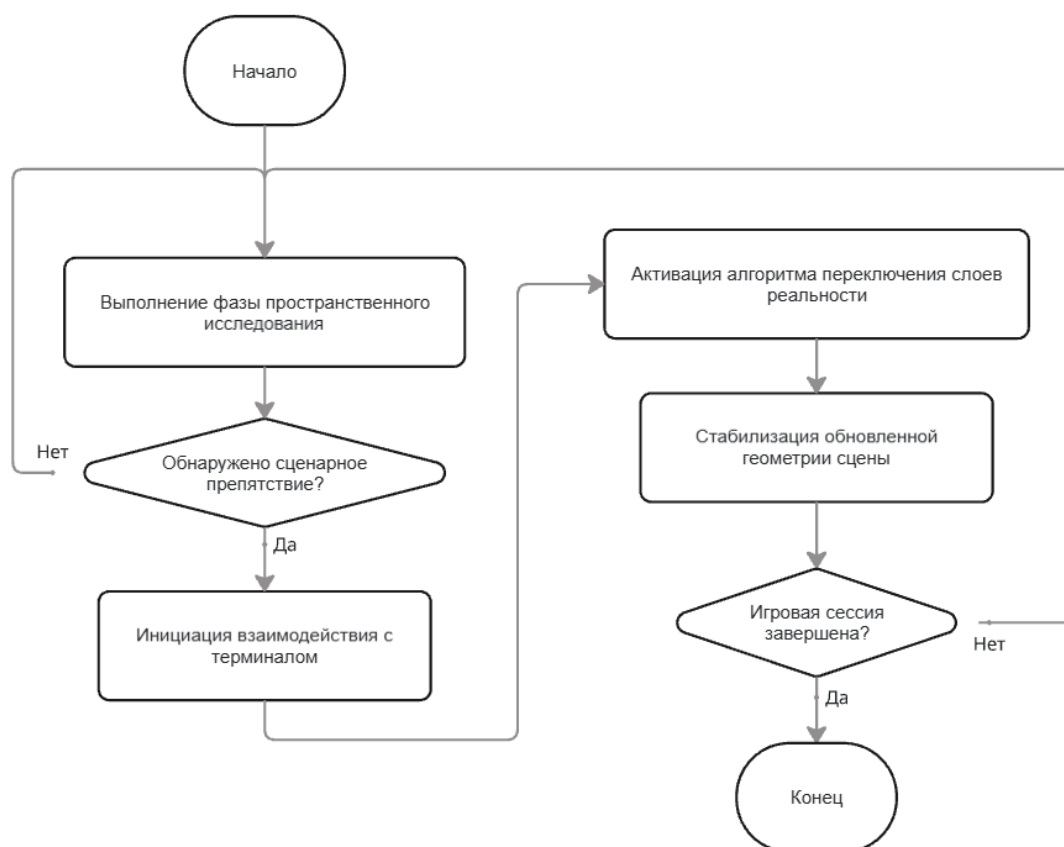


Рисунок 2.2 — Схема функционирования основного игрового цикла.

Подобное проектирование процесса гарантирует, что изменение сценария не является внезапным или неконтролируемым событием. Механика переключения веток органично вплетена в петлю обратной связи: игрок сталкивается с проблемой, анализирует инструмент для ее решения, применяет изменение к миру и немедленно оценивает результат. Подобный подход минимизирует потерю ориентации в пространстве. Пользователь четко осознает причинно-следственную связь между манипуляциями в терминале и последующим изменением окружающего пространства.

Следовательно, формализованный игровой цикл выступает связующим звеном между высокоуровневым замыслом и вычислительными процессами. Он регламентирует опыт пользователя таким образом, чтобы технологически сложные операции по подмене логических массивов воспринимались как естественная и интуитивно понятная механика.

2.3 Проектирование механики переключения веток как инструмента изменения сценария.

Механика переключения веток (Branch Hopping) представляет собой основной инструмент, обеспечивающий динамическую вариативность разрабатываемого игрового процесса. В отличие от классических систем нелинейного повествования, где изменение сценария инициируется посредством выбора реплик в диалоговых деревьях, проектируемое решение интегрирует сценарные развилки непосредственно в топологию виртуального пространства. Процесс перемещения между различными версиями мира функционирует как механизм переписывания правил взаимодействия с окружением, предоставляя пользователю возможность обходить ограничения базового уровня.

Проектирование локаций для поддержки данной механики базируется на концепции одновременного существования нескольких пространственных состояний. В рамках разрабатываемого прототипа архитектура уровней выстраивается вокруг взаимодействия двух ключевых веток репозитория, каждая из которых диктует уникальный сценарий преодоления препятствий. Были выделены следующие характеристики состояний окружения:

- ветка Main характеризуется высокой степенью структурной стабильности, визуальной целостностью архитектурных объектов и наличием информационных подсказок, однако содержит искусственные ограничения в виде непроходимых препятствий и заблокированных дверей;
- ветка Alpha представляет собой измененный массив данных, в котором программные ошибки приводят к искажению законов физики и появлению новых путей (например, элементов геометрии, отсутствующих в базовой версии), что сопровождается появлением враждебных программных алгоритмов и пространственных аномалий.

Таким образом, смена сценарной ветки кардинально трансформирует текущую игровую ситуацию. Пользователь, столкнувшийся с непреодолимой

преградой в стабильной реальности, вынужден переключить контекст выполнения программы в поврежденную среду. Это действие переводит сценарий из фазы безопасного исследования в фазу выживания и тактического обхода угроз. Принятие решения о переключении веток формирует микро-сценарии внутри единой локации, где успех прохождения зависит от аналитического сопоставления рисков и преимуществ каждой из доступных реальностей.

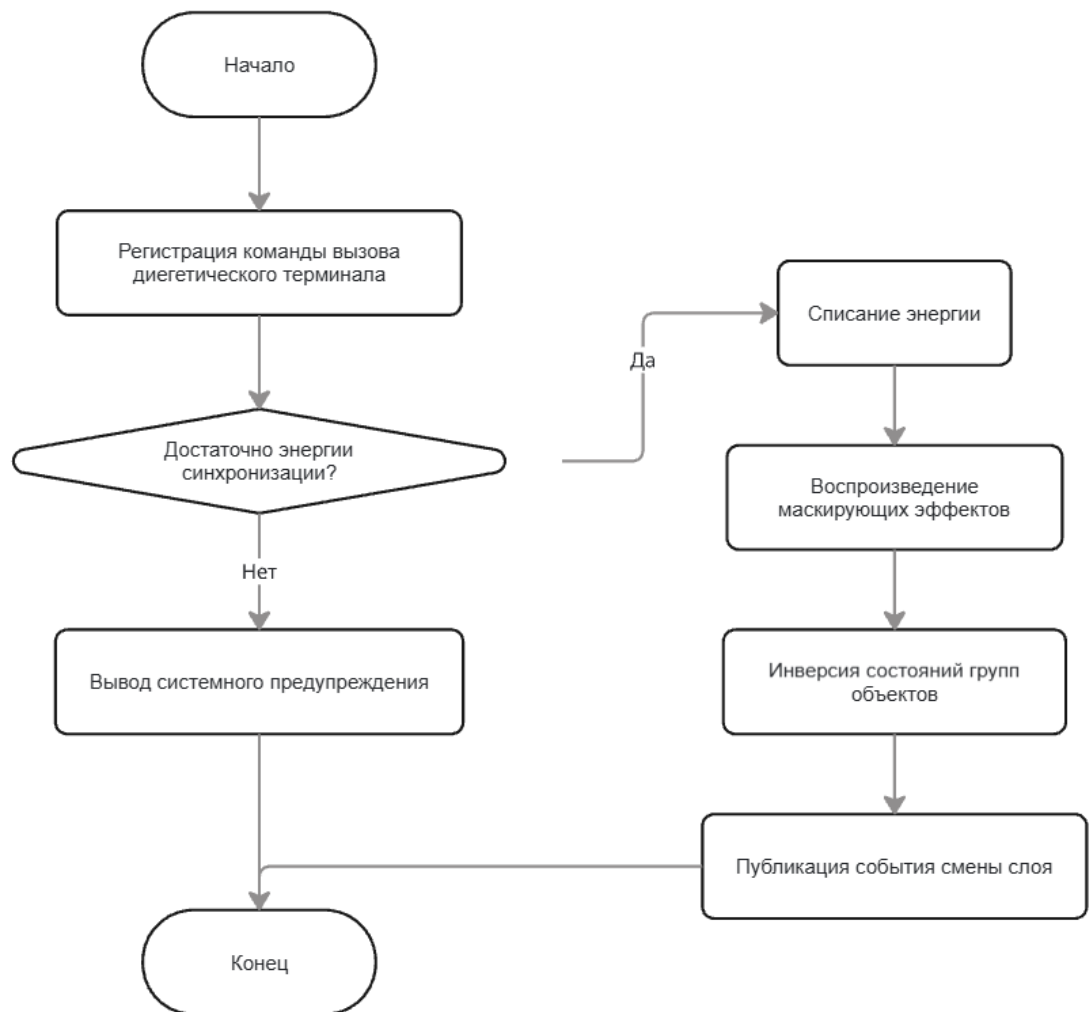


Рисунок 2.3 — Схема процесса динамического переключения слоев реальности

Представленная на рисунке 2.3 схема иллюстрирует, что механика не является простым средством линейного перемещения. Каждое применение терминала генерирует каскад последствий. Открытие нового пути неразрывно связано с активацией дополнительных факторов риска, что требует от игрока

незамедлительного реагирования на изменившиеся условия среды. Следовательно, сценарий уровня не прописывается как единая неизменная последовательность событий, а формируется динамически на основе частоты и своевременности использования алгоритмов трансформации пространства.

Разработка уровней с учетом механики переключения веток накладывает строгие ограничения на архитектурное проектирование. При создании геометрических моделей и расстановке триггеров требуется точное пространственное выравнивание объектов в обеих ветках. В случае несоответствия координат поверхностей существует риск возникновения критических ошибок, при которых персонаж после завершения перехода может оказаться внутри твердотельных структур. Для предотвращения подобных коллизий применяется метод проектирования общих навигационных коридоров с последующей интеграцией изолированных блоков для каждой из версий реальности [2.4, 2.5].

Внедрение описанных принципов в архитектуру приложения позволяет создать глубоко вариативный пользовательский опыт, где инструментарий разработчика программного обеспечения трансформируется в интуитивно понятную игровую механику, напрямую управляющую логикой развития повествования и прохождения препятствий.

Дальнейшая детализация архитектуры уровней требует внедрения концепции якорных объектов и зон абсолютной синхронизации. Якорные объекты представляют собой элементы статической геометрии (например, несущие колонны, серверные стойки или участки пола), координаты и модели столкновений которых остаются строго идентичными и активными во всех ветках репозитория. Данные узлы выполняют функцию пространственного ориентира для пользователя, предотвращая полную потерю ориентации при смене декораций, а также обеспечивают базовую площадь опоры для персонажа в момент осуществления перехода.

Зоны абсолютной синхронизации, в свою очередь, проектируются как специфические помещения, в пределах которых функционал портативного

терминала по смене веток принудительно блокируется на программном уровне. Интеграция таких зон позволяет разработчикам контролировать темп повествования, гарантируя, что игрок окажется в строго заданном состоянии реальности перед началом ключевых сюжетных событий или столкновением с серьезной угрозой. Таким образом, свобода трансформации пространства органично сочетается с заданными узлами повествования.

2.4 Разработка механик взаимодействия: «Code Debugging» и «Anomaly Interaction».

В данном подразделе рассматривается проектирование вспомогательных механик взаимодействия с виртуальным окружением, направленных на углубление процессов исследования и изменения сценарных элементов. В дополнение к глобальной трансформации пространства посредством переключения логических веток, разрабатываемое решение интегрирует локальные методы алгоритмического влияния на конкретные объекты. Ключевыми инструментами такого влияния выступают механики сканирования и устранения ошибок (Code Debugging), а также взаимодействия с пространственными искажениями (Anomaly Interaction).

Механика устранения ошибок представляет собой процесс визуального выявления и последующего исправления программных сбоев в структуре интерактивных объектов. Согласно концепции проекта, многие элементы окружения, особенно локализованные в ветке Alpha, находятся в неработоспособном состоянии вследствие повреждения исходного кода. Для взаимодействия с подобными объектами применяется портативный терминал, функционирующий в режиме диагностического сканирования. Данный режим визуализирует скрытые атрибуты объектов, подсвечивая интерактивные узлы, требующие вмешательства пользователя.

Процесс исправления ошибки формализуется в виде логической мини-игры, где необходимо восстановить правильную последовательность информационных блоков или настроить параметры системы за ограниченное время. Схема алгоритма устранения программных аномалий представлена на рисунке 2.4.

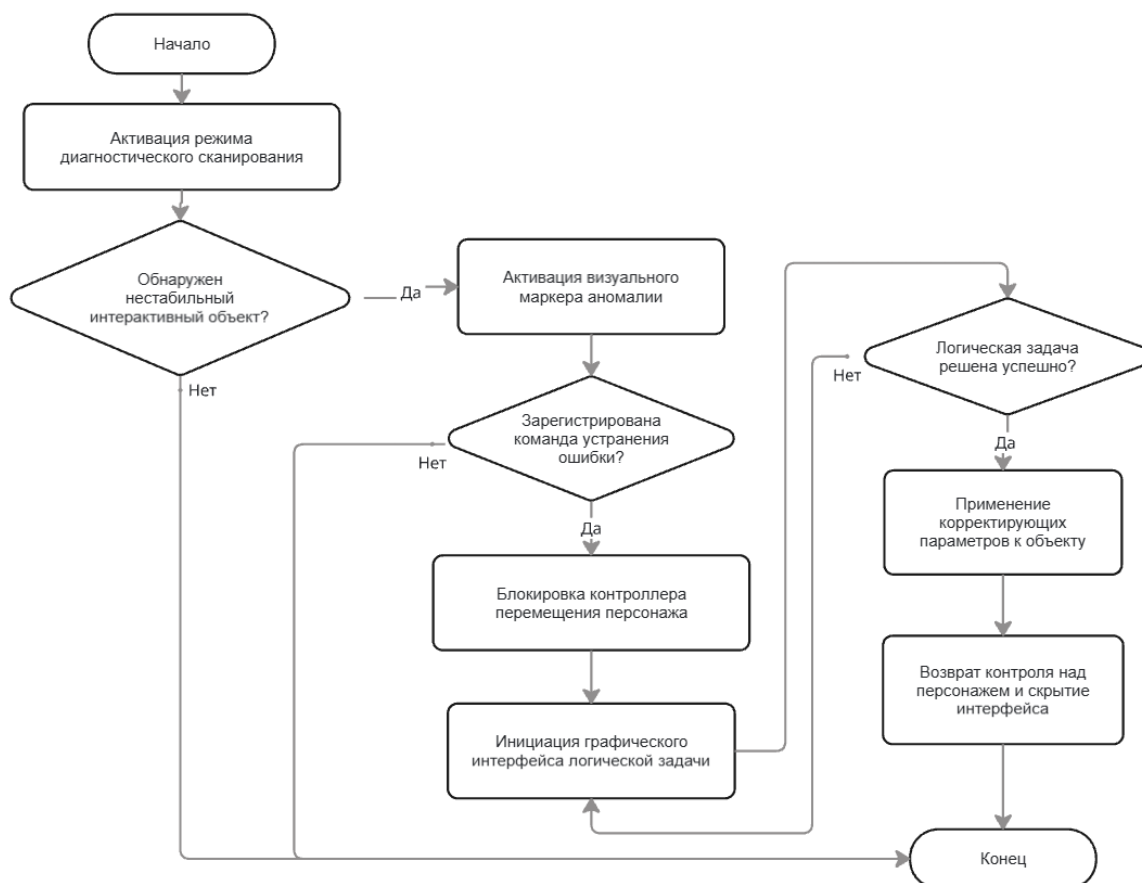


Рисунок 2.4 — Схема алгоритма устранения программных аномалий

Успешное выполнение процедуры отладки немедленно инициирует трансформацию целевого объекта. Таким образом, нефункционирующая преграда может быть преобразована в открытый проход, а опасная зона — в пассивный элемент окружения. Подобный подход позволяет генерировать дополнительные пути преодоления локальных препятствий без необходимости глобального переключения сценарных веток, что существенно расширяет тактический арсенал пользователя.

Параллельно с процессом исправления статических объектов проектируется механика взаимодействия с аномалиями (Anomaly Interaction), регламентирующая поведение пользователя при столкновении с пространственно-временными искажениями. Аномалии представляют собой зоны концентрации критических системных ошибок, возникающие в поврежденных секторах. Нахождение персонажа в радиусе действия аномалии сопряжено с изменением базовых физических законов (например, изменением векторов гравитации или скорости передвижения) и непрерывным снижением параметра стабильности.

Взаимодействие с нестабильными областями базируется на постоянном анализе соотношения рисков и потенциальных преимуществ. Интеграция данной механики в основной игровой цикл обусловлена необходимостью формирования нелинейных маршрутов прохождения. Процесс взаимодействия регламентируется следующими правилами:

- идентификация угрозы. При приближении к границам нестабильной зоны активируются визуальные и звуковые индикаторы, предупреждающие о высокой концентрации ошибок;
- оценка рисков. Пользователь производит анализ целесообразности пересечения аномалии для достижения скрытых данных или сокращения дистанции до целевой точки;
- пересечение нестабильной зоны. В процессе нахождения внутри аномалии инициируется алгоритм постепенного снижения здоровья персонажа, требующий минимизации времени пребывания в опасной среде;
- извлечение выгоды. Успешное преодоление искаженного пространства вознаграждается получением уникальных предметов или информации, необходимых для продвижения по уровню.

Для систематизации внутриигровых угроз и обеспечения разнообразия пользовательского опыта была разработана структурная классификация

пространственных искажений. Аномалии, генерируемые в ветке Alpha, разделяются на три типа, требующих индивидуального подхода к преодолению:

- визуальные и сенсорные искажения: зоны, воздействующие исключительно на отображение виртуальной камеры. Вызывают искажение перспективы, изменение цветов и появление ложных интерактивных маркеров. Преодолеваются посредством использования функции диагностического сканирования терминала, фильтрующей визуальный шум;
- алгоритмические петли: пространственные ловушки, в которых перемещение персонажа принудительно зацикливается, возвращая его в исходную координату. Для нейтрализации требуется выявление и разрушение скрытого узла, инициирующего ошибку, посредством применения сканера;
- деструктивные зоны памяти: области, характеризующиеся отсутствием физической модели пола или наличием опасной среды, наносящей непрерывный урон параметру стабильности персонажа. Пересечение таких зон возможно исключительно при быстром перемещении или использовании особенностей геометрии другой ветки реальности.

Подобное разделение угроз исключает монотонность игрового процесса и стимулирует пользователя к постоянному комбинированию функций устройства для выживания в поврежденной среде [2.6]. Следовательно, комбинация механик отладки кода и взаимодействия с аномалиями формирует комплексную систему взаимодействия пользователя с нестабильной виртуальной средой. Данные алгоритмы позволяют реализовывать вариативность сценария на уровне отдельных объектов и зон локации, дополняя глобальные изменения, обеспечиваемые переключением логических веток. Использование подобных локальных микро-сценариев делает процесс исследования мира непрерывным и требующим постоянной концентрации внимания.

2.5 Проектирование интерфейса (UI/UX) портативного терминала и системы «Data Mining».

Реализация комплексного взаимодействия пользователя с многовариантным окружением требует разработки интуитивно понятного механизма передачи информации. В разрабатываемом программном решении классические элементы двумерного графического интерфейса пользователя минимизированы. Основным инструментом коммуникации выступает портативный терминал, интегрированный непосредственно в трехмерное пространство виртуальной сцены. Подобный подход классифицируется как проектирование внутриигрового интерфейса, что способствует поддержанию высокого уровня погружения и органично вписывается в заданную структуру мира [2.7, 2.8].

Портативный терминал функционирует как универсальный узел управления вычислительными процессами. Архитектура интерфейса базируется на концепции операционной системы, визуализированной посредством голографических проекций, привязанных к модели персонажа. Активация устройства инициирует замедление стандартного цикла перемещения, фокусируя внимание игрока на доступных операциях.

Функционал терминала разделен на следующие логические модули:

- модуль маршрутизации: обеспечивает интерфейс для инициации процесса переключения между стабильной и поврежденной ветками репозитория. Визуализирует текущий статус локации и доступность альтернативных состояний;
- модуль аналитики: управляет параметрами визуального маскирования, активируя эффекты, необходимые для выявления скрытых интерактивных объектов и поврежденных узлов;

- модуль расшифровки: представляет собой интерфейс доступа к текстовым и аудиовизуальным массивам информации, распределенным по уровню, и является центральным элементом механики сбора данных.

Визуальная репрезентация интерфейсных модулей портативного терминала базируется на строгих правилах цветового кодирования и динамической обратной связи. Голографические элементы управления проецируются непосредственно в мировом пространстве, закрепляясь за моделью инструмента на руке персонажа. Для обеспечения высокой степени читаемости в условиях насыщенного и контрастного окружения, базовое состояние интерфейса использует монохромную палитру, стилизованную под устаревшие электронно-лучевые трубки.

При обнаружении интерактивных зон или инициации процессов трансформации цветовая схема терминала алгоритмически изменяется: переход в стабильную ветку сопровождается свечением синего спектра, тогда как подготовка к переключению в поврежденную реальность или обнаружение критической аномалии вызывает пульсирующую красную индикацию. Данная визуальная система дублируется подсистемой звукового дизайна, где каждому действию в терминале присваивается уникальный звуковой сигнал. Комплексное применение визуального и звукового отклика позволяет пользователю осуществлять навигацию по меню и принимать тактические решения без необходимости отведения взгляда от центральной точки взаимодействия с трехмерной сценой [2.9, 2.10, 2.11].

Механика сбора данных выступает основным инструментом реализации нелинейного повествования, не связанного напрямую с физической геометрией локаций. Процесс сбора информации базируется на исследовании скрытых директорий и обнаружении зашифрованных блоков данных (фрагментов кода, системных сообщений). Схема алгоритма извлечения и обработки информационных блоков представлена на рисунке 2.5.

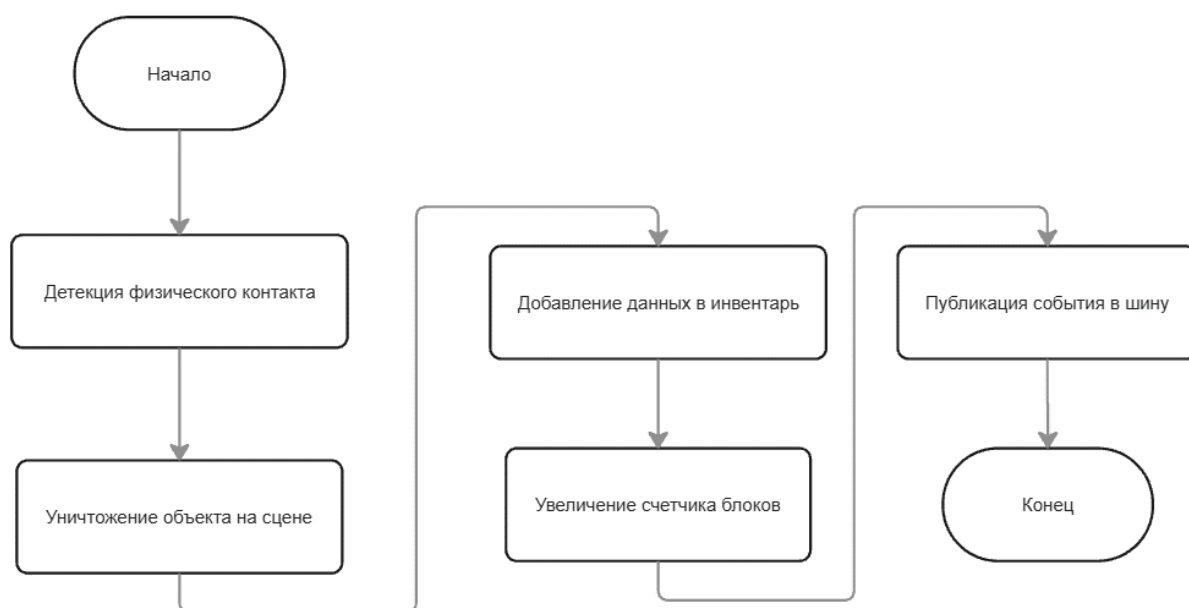


Рисунок 2.6 — Схема алгоритма извлечения и обработки информационных блоков.

Механика сбора данных тесно интегрирована с процессом переключения сценарных веток и функционирует на базе внутриигрового интерфейса терминала. Обновление глобального графа повествования подразумевает, что получение определенных информационных блоков может выступать в качестве условия для активации новых сюжетных событий, открытия ранее заблокированных опций или получения доступа к закрытым областям репозитория. Например, собранные блоки кода из разрушенной реальности могут быть применены в стабильной реальности для программного конструирования отсутствующего моста.

Таким образом, проектирование внутриигрового терминала и системы сбора данных формирует замкнутый информационный контур. Пользователь исследует многовариантное пространство для поиска зашифрованных фрагментов, расшифровка которых посредством терминала открывает доступ к новым сегментам истории и инструментам влияния на мир. Подобный архитектурный подход исключает необходимость применения классических линейных видеороликов, делегируя функцию раскрытия сценария непосредственно на интерактивные системы и аналитические способности игрока. Сформированная в рамках специального раздела проектная

документация, включающая формализацию игрового цикла, алгоритмы переключения веток и устранения ошибок, а также архитектуру подсистемы взаимодействия, составляет исчерпывающую теоретическую и структурную базу для последующей программной реализации прототипа.

Выводы

В специальном разделе дипломной работы было осуществлено комплексное проектирование архитектуры интерактивной системы, базирующейся на механике динамического изменения сценария. На основе сформированных системных требований была разработана концептуальная модель программного продукта Reverse Assembly, использующая парадигму систем контроля версий в качестве фундаментальной структуры виртуального пространства.

В ходе выполнения проектных работ были достигнуты следующие результаты:

- сформирована база многослойного мира, разделенного на изолированные логические состояния (ветки Main и Alpha), что обеспечило платформу для реализации пространственной вариативности и нелинейного преодоления препятствий;
- спроектирована схема алгоритма основного игрового цикла, регламентирующая последовательность действий пользователя от этапа пространственного исследования до алгоритмической трансформации сцены и последующей стабилизации обновленной геометрии;
- детализирована логика функционирования механики переключения веток, выступающей в качестве главного инструмента глобального изменения сценария посредством переключения между альтернативными версиями уровня с учетом концепции якорных объектов;

- разработаны алгоритмы локального взаимодействия с нестабильной виртуальной средой, включающие механики устранения процедурных ошибок в статических объектах и преодоления классифицированных пространственно-временных искажений;
- спроектирован внутриигровой пользовательский интерфейс портативного терминала, интегрированный непосредственно в трехмерную сцену, и формализована подсистема сбора данных, обеспечивающая асинхронное раскрытие сюжетной информации.

Таким образом, разработанная проектная документация, включающая формализацию логических алгоритмов, принципы архитектурного выравнивания объектов и правила балансировки вычислительных ресурсов, полностью удовлетворяет заданным спецификациям. Сформированная структурная база снижает риски возникновения программных коллизий при перестроении геометрии уровня и предоставляет исчерпывающий инструментальный аппарат для последующей технической реализации прототипа разрабатываемого решения в среде движка Unity.

3 ТЕХНОЛОГИЧЕСКИЙ РАЗДЕЛ

В данном разделе будет рассмотрено технологическое, информационное и программное обеспечение разрабатываемой интерактивной системы. Будет приведено детальное описание выбранного инструментального аппарата, структуры базы данных, а также архитектурных паттернов, примененных при конструировании программного кода. Каждая подсистема будет проанализирована с точки зрения ее программной реализации с приведением соответствующих фрагментов исходного кода. В заключительной части главы будут представлены результаты тестирования производительности прототипа при динамической смене слоев виртуального окружения.

3.1 Используемые технологии

В рамках данного подраздела рассматривается выбор программных сред, языков программирования и подключаемых модулей, необходимых для реализации заявленного функционала, а также определяются системные требования к аппаратному обеспечению для корректного функционирования разрабатываемого продукта.

В качестве фундаментальной платформы для разработки интерактивного приложения используется кроссплатформенный игровой движок Unity. Данный выбор обусловлен наличием встроенной компонентно-ориентированной архитектуры, которая позволяет гибко управлять поведением объектов виртуальной сцены. Интеграция среды разработки Unity обеспечивает необходимый инструментальный аппарат для рендеринга трехмерной графики, расчета физических взаимодействий в реальном времени и обработки пользовательского ввода.

Для написания программной логики и реализации архитектурных паттернов применяется объектно-ориентированный язык программирования C#. Статическая типизация и наличие системы автоматической сборки мусора минимизируют вероятность возникновения критических ошибок доступа к памяти в процессе выполнения программы, что является критически важным фактором при разработке систем с динамически изменяемым окружением [3.1, 3.2, 3.3].

Для обеспечения слабой связности компонентов системы и реализации паттерна внедрения зависимостей применяется фреймворк Reflex. Использование данного инструментального аппарата позволяет отказаться от антипаттерна Singleton, делегируя ответственность за создание и предоставление зависимостей специализированному контейнеру. Процесс регистрации базовых сервисов приложения в глобальном контейнере зависимостей представлен в листинге 3.1.

Листинг 3.1 — Фрагмент класса ProjectInstaller, демонстрирующий регистрацию сервисов

```
public void InstallBindings(ContainerBuilder builder)
{
    AudioManager audioManager = Instantiate(_audioManagerPrefab);
    DontDestroyOnLoad(audioManager);
    builder.RegisterValue(audioManager);

    builder.RegisterValue(new
    JsonRepository(Constants.SAVE_FILE_NAME), new
    []{typeof(IPlayerDataRepository)});
    builder.RegisterType(typeof(PlayerPrefsRepository), new
    []{typeof(IDataRepository)}, Lifetime.Singleton, Resolution.Eager);

    builder.RegisterType(typeof(SettingsInteractor),
    Lifetime.Singleton, Resolution.Eager);
    builder.RegisterType(typeof(PlayerDataInteractor),
    Lifetime.Singleton, Resolution.Eager);
}
```

В целях обеспечения персистентности пользовательских данных и сериализации объектов состояния используется библиотека Newtonsoft.Json. Данная библиотека предоставляет высокопроизводительные алгоритмы преобразования сложных иерархических структур данных в текстовый формат JSON и обратно. Использование стороннего сериализатора обосновано необходимостью сохранения вложенных списков и пользовательских классов,

обработка которых стандартными средствами Unity сопряжена с определенными ограничениями. Фрагмент реализации сохранения данных с использованием данной библиотеки представлен в листинге 3.2.

Листинг 3.2 — Фрагмент метода Save класса JsonRepository

```
public void Save<T>(string key, T data)
{
    string timestamp = "01.01.2025-00_00_00";
    string filePath = Path.Combine(Application.persistentDataPath,
    $"_{filePrefix}_{timestamp}.json");

    var dataDict = new SerializableDictionary
    {
        [key] = data
    };

    try
    {
        File.WriteAllText(filePath,
        JsonConvert.SerializeObject(dataDict, Formatting.Indented));
        Debug.Log($"Saved data to {filePath}");
    }
    catch (Exception ex)
    {
        Debug.LogError($"Failed to save to {filePath}:
        {ex.Message}");
    }
}
```

Для реализации плавных переходов графического интерфейса пользователя и процедурных анимаций применяется библиотека DOTween. Интеграция данного программного решения позволяет создавать комплексные анимационные последовательности без использования встроенного компонента Animator, что существенно снижает вычислительную нагрузку на центральный процессор и упрощает алгоритмическое управление визуальными эффектами. В листинге 3.3 продемонстрировано применение библиотеки для анимации интерфейса сканирующего устройства.

Листинг 3.3 — Фрагмент класса ScannerView с использованием методов DOTween

```
public void FillIn()
{
    _mask.DOSizeDelta(_maskSize, _fillDuration);
}

public void FillOut()
{
    _mask.DOSizeDelta(Vector2.zero, _fillDuration);
}
```

3.2 Информационное обеспечение

Разработка комплексных интерактивных систем требует проектирования надежной подсистемы хранения и обработки информации. В данном подразделе будет рассмотрена структура данных, используемая для фиксации глобальных настроек и сериализации игровых сессий, а также программные алгоритмы, обеспечивающие чтение и запись массивов информации. Архитектура информационного обеспечения выстроена с применением паттерна проектирования Repository, что гарантирует изолированность бизнес-логики от конкретных механизмов долговременного хранения данных.

3.2.1 Описание классов для хранения данных

Для обеспечения персистентности (сохранности состояния между запусками приложения) были спроектированы сериализуемые модели данных. Выделяются две основные категории хранимой информации: глобальные пользовательские настройки и контекстные данные конкретной игровой сессии [3.6, 3.7, 3.8].

Для хранения глобальных параметров программного продукта был разработан класс `GameSettings`. Структура данного класса включает в себя конфигурацию уровня сложности, выраженную через множитель характеристик противников, а также логический флаг, определяющий активность режима мирного исследования локаций. Программное описание модели настроек представлено в листинге 3.4.

Листинг 3.4 — Структура класса `GameSettings`

```
[Serializable]
public class GameSettings
{
    public float EnemiesPower = 1f;
    public bool PeaceMode;
}
```

Сохранение прогресса прохождения требует фиксации множества переменных, описывающих текущее состояние виртуального мира. Для этих целей был спроектирован класс `PlayerData`, агрегирующий в себе информацию о цифровом аватаре и его окружении. В структуру включены количественный показатель стабильности (здоровья), вектор координат расположения в трехмерном пространстве, счетчик собранных информационных фрагментов (блоков кода), а также массив данных об активных противниках на уровне.

Информация о противниках вынесена в отдельную модель `EnemyData`. Хранение индекса префаба (шаблона объекта) позволяет вычислительной системе при загрузке сохранения инстанцировать (создать экземпляр) корректный тип противника, а уникальный идентификатор и позиция гарантируют точное восстановление сценарной обстановки. Структура классов, описывающих состояние сессии, приведена в листинге 3.5.

Листинг 3.5 — Структуры классов `PlayerData` и `EnemyData`

```
[Serializable]
public class PlayerData
{
    public float Health;
    public Vector3 Position;
    public List<EnemyData> Enemies;
    public int CodeBlocks;
}

[Serializable]
public class EnemyData
{
    public string Id;
    public float Health;
    public Vector3 Position;
    public int PrefabIndex;
    public bool IsBoss;
}
```

3.2.2 Программная реализация операций с данными

Архитектура подсистемы ввода-вывода базируется на использовании интерфейса `IDataRepository`, который декларирует контракты для стандартных операций: сохранения, загрузки, проверки наличия ключа и удаления данных.

Для управления массивами игровых сессий данный интерфейс был расширен до `IPlayerDataRepository`, в который были добавлены алгоритмы обработки временных меток (таймстемпов).

В качестве механизма хранения глобальных настроек был выбран класс `PlayerPrefsRepository`, использующий встроенный функционал движка Unity. Данный подход обоснован малым объемом конфигурационных данных. Метод сохранения производит проверку типа входящего значения посредством конструкции переключателя и вызывает соответствующий метод системного интерфейса, что показано в листинге 3.6.

Листинг 3.6 — Фрагмент реализации метода `Save` в классе `PlayerPrefsRepository`

```
public void Save<T>(string key, T data)
{
    switch (data)
    {
        case int valueInt:
            PlayerPrefs.SetInt(key, valueInt);
            break;
        case float valueFloat:
            PlayerPrefs.SetFloat(key, valueFloat);
            break;
        case string valueString:
            PlayerPrefs.SetString(key, valueString);
            break;
        case bool valueBool:
            PlayerPrefs.SetInt(key, valueBool ? 1 : 0);
            break;
        default:
        {
            string json = JsonUtility.ToJson(data);
            PlayerPrefs.SetString(key, json);
            break;
        }
    }

    PlayerPrefs.Save();
}
```

Для хранения комплексных структур игровых сессий был разработан класс `JsonRepository`. Алгоритм сохранения в данном классе подразумевает генерацию уникального текстового файла для каждой сессии с добавлением временной метки в имя файла. При инициации загрузки алгоритм осуществляет поиск всех файлов с заданным префиксом, сортирует их в хронологическом порядке и извлекает информацию из наиболее актуального архива либо из файла с явно указанным идентификатором времени.

Связующим звеном между репозиториями и остальными системами приложения выступают классы-интеракторы: `SettingsInteractor` и `PlayerDataInteractor`. Их назначение заключается в инкапсуляции ключей доступа к базе данных и предоставлении высокоуровневого интерфейса для других скриптов. Так, класс `PlayerDataInteractor` хранит в оперативной памяти ссылку на текущую загруженную сессию в свойстве `CurrentSave`, что минимизирует количество дорогостоящих обращений к файловой системе при чтении координат или инвентаря игрока. Алгоритм инициализации новой игры и сохранения текущего прогресса внутри интерактора продемонстрирован в листинге 3.7.

Листинг 3.7 — Фрагмент класса `PlayerDataInteractor`

```
public void StartNewGame()
{
    CurrentSave = new PlayerData
    {
        Health = 100f,
        Position = default,
        Enemies = new List<EnemyData>()
    };
    _playerDataRepository.Save(PlayerDataKey, CurrentSave);
}

public void SavePlayerData(PlayerData playerData)
{
    CurrentSave = playerData;
    _playerDataRepository.Save(PlayerDataKey, CurrentSave);
}
```

Таким образом, спроектированное информационное обеспечение гарантирует отказоустойчивое сохранение данных различных типов. Применение паттерна `Repository` и классов-интеракторов обеспечивает высокую степень модульности программного кода, позволяя при необходимости произвести замену формата сериализации или целевого носителя информации без внесения критических изменений в логику игрового процесса.

3.3 Программное обеспечение

Проектирование сложных интерактивных систем с множеством взаимосвязанных механик требует формирования надежного фундамента. В данном подразделе детально рассматриваются базовые паттерны проектирования, обеспечивающие модульность, масштабируемость и отказоустойчивость разрабатываемого программного продукта. Основной акцент сделан на механизмах инициализации, управлении зависимостями, а также на внутрисистемной маршрутизации данных и управлении поведением сущностей.

3.3.1 Инициализация сессии, перемещение и первичная смена слоя реальности

Разработка программного продукта осуществлялась на базе кроссплатформенного игрового движка Unity с использованием объектно-ориентированного языка C#. Для обеспечения слабой связности компонентов и централизованного управления зависимостями был применен паттерн Inversion of Control (IoC) на базе фреймворка Reflex, а также реализована паттерн-шина событий (EventBus).

Игровой процесс начинается с инициализации виртуальной среды и появления цифрового аватара. За сборку персонажа отвечает класс `PlayerBootstrap`, реализующий контракт `IBootstrapComponent`. В процессе выполнения метода `Boot` происходит инстанцирование префаба, внедрение зависимостей и публикация глобального события `PlayerSpawnEvent`, уведомляющего остальные системы об успешном создании игрока. Программная реализация данного этапа представлена в листинге 3.1.

Листинг 3.1 — Фрагмент инициализации персонажа в классе `PlayerBootstrap`

```
private void SetupPlayer()  
{
```

```

        Player = Instantiate(_playerPrefab,
        _playerSpawnPoint.position, Quaternion.identity);
        GameObjectInjector.InjectRecursive(Player, _container);
        _camera = Player.GetComponentInChildren<Camera>();
        _playerStabilitySystem =
        Player.GetComponent<StabilitySystem>();

        // Инициализация контроллеров
        var movementController =
        Player.GetComponent<MovementController>();
        var skillsController =
        Player.GetComponent<SkillsController>();
        skillsController.Init(_camera);

        EventBus.Raise(new PlayerSpawnEvent()
        {
            PlayerTransform = Player.transform,
            Camera = _camera
        });
    }

```

После появления на уровне осуществляется перемещение аватара по стартовому коридору. Логика передвижения контролируется классом `MovementController`, функционирующим на основе паттерна «Конечный автомат» (State Machine). Переход из состояния бездействия (IdleState) в состояние ходьбы (WalkingState) инициируется при получении ненулевого вектора направления от подсистемы `InputManager`.

Продвижение по коридору прерывается тупиком — глухой стеной с всплывающей информационной подсказкой. На данном этапе происходит первичное применение механики динамического изменения сценария. Для преодоления преграды используется портативный терминал, визуализированный на руке персонажа. Обработка ввода и вызов интерфейса устройства делегированы классу `WristTerminalController`.

Листинг 3.2 — Вызов диалектического терминала в классе `WristTerminalController`

```

public void ToggleTerminal()
{
    if (_terminalView == null)
    {
        _terminalView = _viewManager.GetView<TerminalView>();
    }

    IsTerminalOpen = !IsTerminalOpen;

    if (IsTerminalOpen)
    {
        _terminalView.UpdateInfo(_branchManager.CurrentBranch);
        _terminalView.Show();
    }
}

```

```

        else
        {
            _terminalView.Hide();
        }
    }
}

```

Активация терминала выводит в мировые координаты диегетический графический интерфейс (TerminalView). Посредством нажатия соответствующей клавиши передается команда на переключение текущей версии виртуального мира. Сигнал обрабатывается навыком SwitchBranchSkill, который обращается к классу BranchManager для инициации трансформации. Алгоритм смены слоев представлен в листинге 3.3.

Листинг 3.3 — Метод SwitchBranch в классе BranchManager

```

public void SwitchBranch(WorldBranch branch, bool instant = false)
{
    CurrentBranch = branch;

    if (_mainBranchEnv != null)
        _mainBranchEnv.SetActive(CurrentBranch == WorldBranch.Main);
    if (_alphaBranchEnv != null)
        _alphaBranchEnv.SetActive(CurrentBranch == WorldBranch.Alpha);

    if (!instant)
    {
        if (_flashCanvas != null)
        {
            _flashCanvas.alpha = 1f;
            _flashCanvas.DOFade(0f, 0.5f).SetEase(Ease.OutQuad);
        }

        if (Camera.main != null)
        {
            Camera.main.DOComplete();
            Camera.main.DOShakePosition(0.4f, 0.3f, 20, 90f,
false, ShakeRandomnessMode.Harmonic);
        }

        if (_audioSource != null && _jumpSound != null)
        {
            _audioSource.PlayOneShot(_jumpSound);
        }
    }

    EventBus.Raise(new BranchSwitchedEvent(CurrentBranch));
}

```

В процессе выполнения метода SwitchBranch происходит деактивация иерархии объектов текущего слоя и активация целевого. Для сохранения эффекта погружения мгновенная подмена геометрии маскируется аудиовизуальными эффектами средствами библиотеки DOTween: вспышкой интерфейса и процедурным сотрясением камеры. В результате стена, блокировавшая путь в

начальной ветке, исчезает, так как она отсутствует в топологии альтернативной ветки репозитория, открывая проход для дальнейшего исследования.

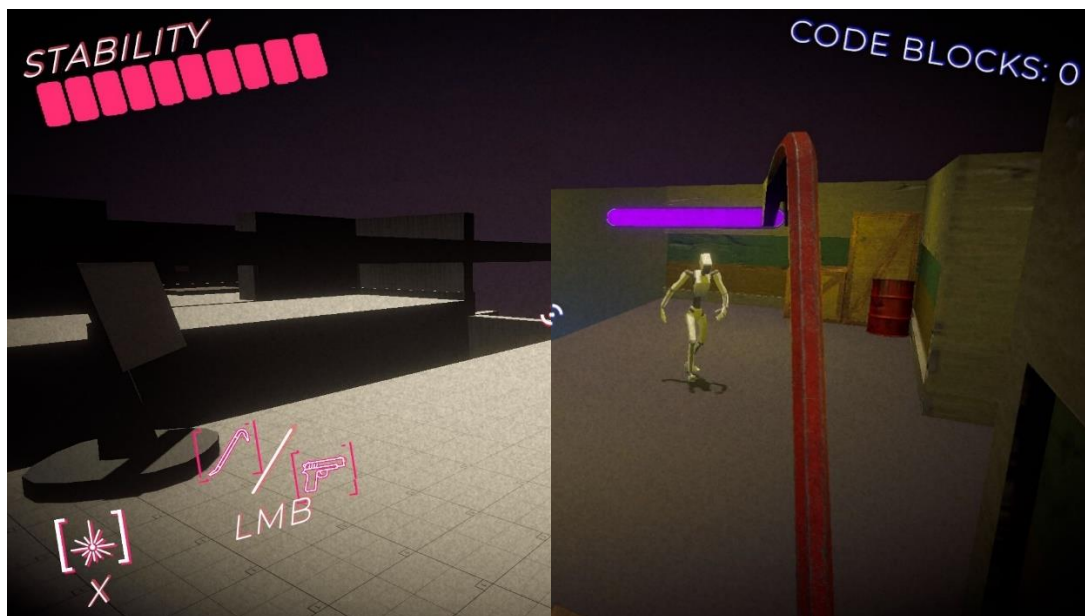


Рисунок 3.1 — Визуальное сравнение топологии локации в слоях Main (слева) и Alpha (справа)

3.3.2 Система диагностического сканирования и определение отсутствующих компонентов

После успешного переключения слоя реальности и обхода пространственного препятствия дальнейший маршрут приводит к стационарному терминалу. Взаимодействие с данным объектом напрямую заблокировано, в связи с чем требуется применение системы диагностического сканирования для выявления причин неисправности.

Активация режима сканирования осуществляется через специализированный навык ScannerSkill. При вызове данной способности происходит изменение визуального представления сцены с помощью компонента ScannerView, накладывающего на экран графическую маску. Параллельно запускается алгоритм ежекадрового поиска интерактивных объектов, реализованный посредством пуска математического луча (Raycast) из центральной точки виртуальной камеры.

Для оптимизации вычислительной нагрузки проверка пересечений луча строго ограничена определенной битовой маской слоев (InteractableLayer), что исключает обработку статической геометрии уровня (стен и пола). Логика работы алгоритма сканирования представлена в листинге 3.4.

Листинг 3.4 — Метод OnTick в классе ScannerSkill

```
protected override void OnTick()
{
    if (!IsScannerActive) return;

    // Рейкаст из центра экрана
    if (Physics.Raycast(_cameraTransform.position,
        _cameraTransform.forward, out RaycastHit hit,
        _def.ScanDistance, _def.InteractableLayer))
    {
        var buggable = hit.collider.GetComponent<IBuggable>();

        if (buggable != null && buggable.IsBugged)
        {
            if (_currentTarget != buggable)
            {
                _currentTarget?.OnScanned(false);
                _currentTarget = buggable;
                _currentTarget.OnScanned(true); // Подсвечиваем
красным
            }
        }
    }
    else
    {
        if (_currentTarget != null)
        {
            _currentTarget.OnScanned(false);
            _currentTarget = null;
        }
    }
}
```

Как демонстрирует приведенный фрагмент кода, система сканирования реагирует исключительно на объекты, реализующие интерфейс IBuggable. Использование единого интерфейса позволяет стандартизировать процесс отладки для различных элементов окружения, обеспечивая модульность архитектуры. Структура данного контракта описывается четырьмя ключевыми методами и свойствами, представленными в листинге 3.5.

Листинг 3.5 — Интерфейс IBuggable

```
namespace Gameplay.Interactables
{
    public interface IBuggable
    {
        bool IsBugged { get; }
    }
}
```

```
        void OnScanned(bool isScanning);  
        void OnInteract();  
        void FixBug();  
    }  
}
```

При успешном обнаружении стационарного терминала лучом сканера, система вызывает метод `OnScanned(true)`. В результате целевой объект визуально выделяется (посредством активации компонента `Outline` и смены цвета контура на красный), информируя о наличии технической ошибки или недостающих данных.

Последующее нажатие клавиши взаимодействия инициирует выполнение метода `OnInteract()`. На данном этапе система временно блокирует элементы управления персонажем, освобождает курсор и выводит на экран графический интерфейс (посредством обращения к классу `ViewManager`). Анализ выведенного интерфейса позволяет установить причину неисправности: для восстановления работоспособности терминала и дальнейшего продвижения требуется наличие одного информационного фрагмента (блока кода), который в инвентаре пользователя на данный момент отсутствует.

3.3.3 Боевая система, устранение противников и сбор информационных фрагментов

Для получения требуемого информационного фрагмента осуществляется повторное применение портативного терминала и переход в альтернативную (поврежденную) версию виртуального мира. В данной реальности генерируются агрессивные алгоритмы защиты (противники), препятствующие свободному перемещению. Логика поведения противников контролируется классом `EnemyController`, функционирующим на базе конечного автомата.

Взаимодействие с противниками опирается на модульную боевую систему. Пользователю предоставляется возможность использования дистанционных и ближних атак, обрабатываемых компонентом `SkillsController`. Для нейтрализации

угрозы применяется дистанционный выстрел (класс `ProjectileSkill`), генерирующий физический снаряд. При столкновении снаряда с моделью противника алгоритм не только регистрирует нанесение урона, но и проверяет наличие интерфейса `IGlitchable`, отвечающего за применение эффекта алгоритмического оглушения. Программная реализация обработки коллизии снаряда представлена в листинге 3.6.

Листинг 3.6 — Метод `OnTriggerEnter` в классе `Projectile`

```
private void OnTriggerEnter(Collider other)
{
    IDamageable damageable =
other.gameObject.GetComponent<IDamageable>();
    if (damageable != null && damageable != _self)
    {
        DoDamage(damageable);

        var glitchable =
other.gameObject.GetComponent<IGlitchable>();
        glitchable?.ApplyGlitchStun(3f);
    }

    if (damageable != _self)
    {
        Destroy(gameObject);
    }
}
```

При вызове метода `ApplyGlitchStun` конечный автомат противника перехватывает изменение логического флага и принудительно переводится в состояние `GlitchStunState`. В данном состоянии блокируется алгоритм поиска пути (`NavMeshAgent`), а скорость воспроизведения анимации снижается до нуля, что визуально имитирует зависание программного кода. Подобная механика предоставляет персонажу тактическое преимущество для безопасного сближения и нанесения последующего урона в ближнем бою.

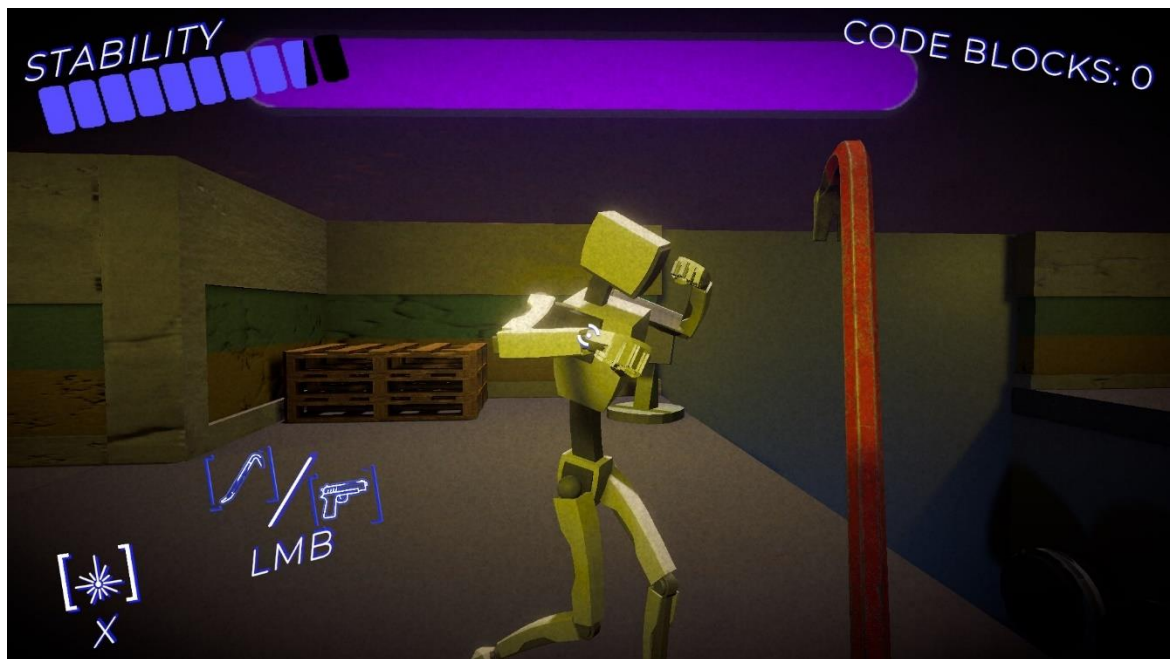


Рисунок 3.2 — Противник

Физический расчет урона интегрирован с компонентом `StabilitySystem`. При падении показателя стабильности противника до нулевого значения вызывается событие смерти. В этот момент конечный автомат переходит в состояние `DeathState`, в рамках которого блокируются компоненты физического столкновения, инициируется анимация уничтожения и вызывается метод `SpawnLoot()`. Данный метод инстанцирует (создает на сцене) на месте поверженного врага физический объект — информационный фрагмент (блок кода).

Процесс сбора полученного предмета реализован через компонент `CodeBlockPickup`. При вхождении персонажа в зону триггера объект уничтожается, а в глобальную шину транслируется специализированное событие `CodeBlockCollectedEvent`. Данный механизм гарантирует, что объект инвентаря не будет зарегистрирован дважды. Реализация сбора продемонстрирована в листинге 3.7.

Листинг 3.7 — Метод `OnTriggerEnter` в классе `CodeBlockPickup`

```
private void OnTriggerEnter(Collider other)
{
    if (other.CompareTag("Player"))
    {
        EventBus.Raise(new CodeBlockCollectedEvent());
    }
}
```

```
        Destroy(gameObject);  
    }  
}
```

Глобальная маршрутизация сбора данных осуществляется классом `InventoryManager`. Данный компонент подписывается на событие сбора информационного блока в момент инициализации сцены. При перехвате события происходит инкрементация (увеличение) внутреннего счетчика инвентаря и отправка команды на обновление текстового индикатора в графическом интерфейсе пользователя (`PlayerHUDView`). Алгоритм обновления данных инвентаря приведен в листинге 3.8.

Листинг 3.8 — Фрагмент обработки сбора данных в классе `InventoryManager`

```
private void OnBlockCollected(CodeBlockCollectedEvent data)  
{  
    _codeBlocks++;  
    UpdateHUD();  
}  
  
private void UpdateHUD()  
{  
    if (_hudView != null)  
    {  
        _hudView.UpdateBlocksCount(_codeBlocks);  
    }  
}
```

В результате успешного боевого взаимодействия инвентарь пополняется необходимым ресурсом, что открывает возможность для алгоритмического восстановления преграждающих конструкций и продолжения исследования виртуального комплекса.



Рисунок 3.3 — Визуализация информационного фрагмента (блока кода) и индикатора инвентаря пользователя

3.3.4 Алгоритмическое конструирование объектов и пространственная навигация

После успешного сбора необходимого информационного фрагмента осуществляется повторное применение портативного терминала для возврата в первоначальную, стабильную ветку реальности. Данное действие обусловлено тем, что взаимодействие со стационарным терминалом управления мостом доступно только в неповрежденной версии виртуальной среды, где отсутствует агрессивный программный код противников.

Приближаясь к стационарному терминалу, пользователь инициирует процесс взаимодействия. Для корректной работы с графическими интерфейсами, требующими использования курсора, система должна временно переопределить контекст управления. В момент нажатия клавиши взаимодействия архитектура приложения обращается к подсистеме InputManager для блокировки потока команд, отвечающих за пространственное перемещение и применение навыков

персонажа. Программная реализация данного механизма представлена в листинге 3.9.

Листинг 3.9 — Методы блокировки и разблокировки ввода в классе InputManager

```
public void DisablePlayerInput()
{
    _playerActions.Disable();
    MoveInput = Vector2.zero;
    SprintInput = false;
}

public void EnablePlayerInput()
{
    _playerActions.Enable();
}
```

Блокировка действий обнуляет векторы движения, предотвращая неконтролируемое перемещение аватара во время работы с меню. После отключения ввода на экран выводится графический интерфейс стационарного терминала (посредством ViewManager), а курсор мыши разблокируется для взаимодействия с элементами управления.

В графическом интерфейсе терминала отображается требование о наличии одного собранного блока кода. Поскольку на предыдущем этапе требуемый фрагмент был добавлен в инвентарь и зафиксирован классом InventoryManager, пользователь имеет возможность нажать активную кнопку подтверждения на экране.

Нажатие кнопки запускает логику конструирования переправы: недостающий геометрический массив моста активируется в пространстве сцены. Сразу после этого интерфейс терминала закрывается, а система возвращает контроль над персонажем. Базовая логика возврата управления инкапсулирована в абстрактном классе PuzzleViewBase, фрагмент которого продемонстрирован в листинге 3.10.

Листинг 3.10 — Метод завершения взаимодействия в классе PuzzleViewBase

```
public virtual void ClosePuzzle()
{
    Hide();
    _currentTarget = null;

    _inputManager.EnablePlayerInput();
    Cursor.lockState = CursorLockMode.Locked;
    Cursor.visible = false;
}
```

Как видно из представленного кода, метод ClosePuzzle не только скрывает графическое окно (Hide()), но и восстанавливает привязку курсора к центру экрана, одновременно активируя методы перемещения персонажа.

Несмотря на успешное появление моста в стабильной реальности, прямое прохождение по нему блокируется новым препятствием. Для преодоления сгенерированной переправы пользователь вынужден вновь применить портативный терминал и инициировать переход в поврежденную ветку реальности. В слое Alpha структура моста имеет измененную топологию, позволяющую персонажу беспрепятственно пересечь пропасть и продолжить движение по коридору до следующей информационной подсказки.

Таким образом, на данном этапе прохождения раскрывается комплексный характер взаимодействия с виртуальной средой: решение одной задачи требует от пользователя многократного переключения между слоями реальности, поиска ресурсов и использования внутриигровых терминалов для алгоритмической сборки пути.

3.3.5 Алгоритмическая отладка объектов и решение логических головоломок

После успешного пересечения сконструированного моста дальнейшее продвижение персонажа ограничивается новым сценарным препятствием — заблокированным проемом, перекрытым неактивной преградой (планкой). В стабильной версии виртуальной среды механизмы взаимодействия с данным препятствием отсутствуют. Для выявления скрытых алгоритмических уязвимостей осуществляется очередное применение портативного терминала, инициирующее переход в поврежденную ветку реальности.

В альтернативном слое геометрия преграды содержит программную ошибку, для обнаружения которой применяется система диагностического

сканирования. При наведении сканера нестабильный объект распознается системой как сущность, требующая алгоритмической отладки. Взаимодействие с поврежденным элементом активирует логическую головоломку, классифицируемую как пересборка информационных блоков (коммитов).



Рисунок 3.4 — Активация режима диагностического сканирования и детекция нестабильного объекта

Данная головоломка выводится на экран посредством класса CommitPuzzleView, функционирующего поверх основного игрового процесса. Интерфейс задачи представляет собой набор графических элементов, имитирующих строки программного кода. Для восстановления работоспособности преграды пользователю необходимо выстроить представленные строки в логически верной последовательности.

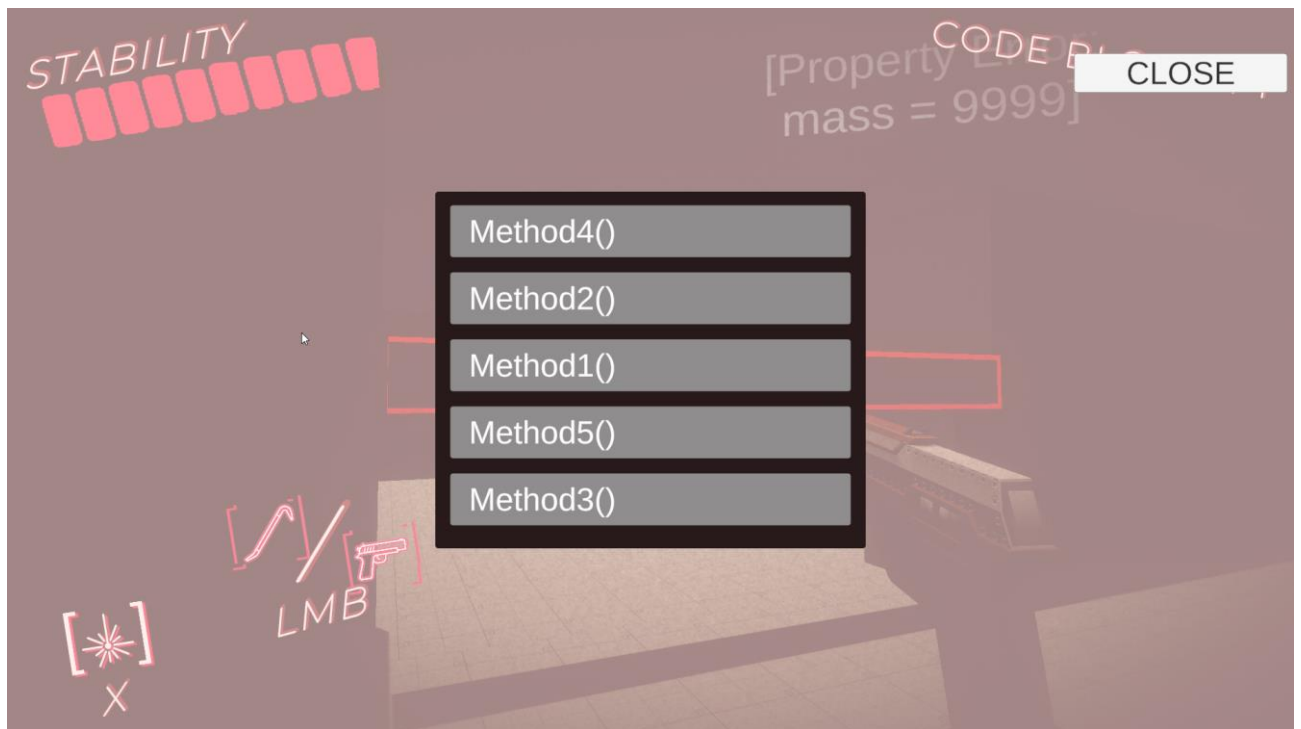


Рисунок 3.5 — Графический интерфейс процесса алгоритмической отладки физических свойств объекта

Перемещение графических элементов реализуется за счет обработки событий перетаскивания (Drag-and-Drop) пользовательского интерфейса. Каждый перемещаемый блок управляется компонентом `DraggableCodeLine`, который перехватывает координаты курсора мыши и динамически вычисляет новую позицию элемента в иерархии списка (используя параметр `SiblingIndex`). Алгоритм расчета позиции при перетаскивании представлен в листинге 3.11.

Листинг 3.11 — Метод `OnDrag` в классе `DraggableCodeLine`

```
public void OnDrag(PointerEventData eventData)
{
    transform.position = eventData.position;

    int newSiblingIndex = _parent.childCount - 1;

    for (int i = 0; i < _parent.childCount; i++)
    {
        Transform sibling = _parent.GetChild(i);
        if (sibling == this.transform || sibling ==
        _placeholder.transform) continue;

        if (eventData.position.y > sibling.position.y)
        {
            newSiblingIndex = sibling.GetSiblingIndex();
            if (_placeholder.transform.GetSiblingIndex() <
            newSiblingIndex)
            {
```

```

        newSiblingIndex--;
    }
    break;
}
}

_placeholder.transform.SetSiblingIndex(newSiblingIndex);
}

```

Как видно из представленного листинга, в процессе перетаскивания создается временный графический объект (плейсхолдер), который резервирует место в списке, предотвращая смещение остальных элементов. Вычислительная система циклично проверяет вертикальные координаты курсора относительно соседних блоков и обновляет индекс плейсхолдера, обеспечивая плавную и предсказуемую сортировку списка.

По завершении операции перетаскивания (отпускании клавиши мыши) вызывается событие `OnDragEnded`, которое перехватывается основным классом головоломки `CommitPuzzleView`. В этот момент иницируется алгоритм проверки условия победы. Система последовательно сравнивает текущие индексы расположения строк кода с заранее заданным эталонным массивом `_correctOrder`. Программная реализация проверки валидности собранного кода продемонстрирована в листинге 3.12.

Листинг 3.12 — Метод `CheckWinCondition` в классе `CommitPuzzleView`

```

private void CheckWinCondition()
{
    if (_correctOrder == null || _correctOrder.Length == 0)
return;

    for (int i = 0; i < _correctOrder.Length - 1; i++)
    {
        if (_correctOrder[i] != null && _correctOrder[i+1] !=
null)
        {
            if (_correctOrder[i].GetSiblingIndex() >
_correctOrder[i+1].GetSiblingIndex())
            {
                return;
            }
        }
    }

    foreach (var line in _codeLines)
    {
        if (line != null) line.OnDragEnded -= CheckWinCondition;
    }

    OnWin();
}

```

В случае если каждая последующая строка располагается ниже предыдущей (индекс элемента в иерархии больше), цикл проверки успешно завершается. При подтверждении правильности собранной последовательности вызывается базовый метод OnWin, который скрывает графический интерфейс, возвращает управление цифровому аватару и удаленно активирует метод FixBug на целевом объекте.

Выполнение процедуры отладки восстанавливает корректную работоспособность преграды: визуальные индикаторы программной ошибки деактивируются, а пространственный проем открывается, предоставляя возможность беспрепятственного прохождения в следующую секцию виртуального уровня.

3.3.6 Отладка физических свойств объектов и синхронизация параметров

Продвижение в следующую секцию виртуального уровня приводит персонажа к финальному сценарному узлу. Пространство блокируется массивным геометрическим объектом (кубом), расположенным над специализированным триггерным механизмом (нажимной плитой). Особенностью данной конфигурации является то, что куб находится в подвешенном состоянии с нарушенным вектором гравитации и аномальными физическими характеристиками, препятствующими его естественному взаимодействию с окружением.

Для устранения возникшей пространственной аномалии применяется система диагностического сканирования. Идентификация нестабильного объекта и последующее взаимодействие с ним инициируют вызов графического интерфейса, предназначенного для ручной калибровки физических параметров сущности. Данный процесс формализован в виде мини-игры «Синхронизация физики», управление которой делегировано классу PhysicsPuzzleView.

Интерфейс головоломки представляет собой панель управления, оснащенную тремя динамическими индикаторами (ползунками). Задачей пользователя является балансировка значений каждого из трех параметров таким образом, чтобы они одновременно находились в строго заданном целевом диапазоне (в «зеленой зоне»). Для повышения вариативности прохождения начальные значения индикаторов генерируются процедурно при каждом вызове интерфейса с использованием класса `Random`. Алгоритм инициализации задачи представлен в листинге 3.13.

Листинг 3.13 — Метод `ShowPuzzle` в классе `PhysicsPuzzleView`

```
public override void ShowPuzzle(IBuggable target)
{
    base.ShowPuzzle(target);

    foreach (var slider in _sliders)
    {
        if (slider == null) continue;
        slider.value = Random.Range(0f, 1f);
        slider.onValueChanged.RemoveAllListeners();
        slider.onValueChanged.AddListener(CheckWinCondition);
    }

    if (_applyButton != null)
    {
        _applyButton.interactable = false;
        _applyButton.onClick.RemoveAllListeners();
        _applyButton.onClick.AddListener(OnWin);
    }

    CheckWinCondition(0f);
}
```

Как следует из представленного программного кода, любые манипуляции пользователя с индикаторами перехватываются подсистемой событий графического интерфейса (`onValueChanged`) и инициируют вызов метода `CheckWinCondition`. Данный метод осуществляет математическую проверку текущих значений на принадлежность целевому диапазону, ограниченному переменными `_targetMin` и `_targetMax`. Логика валидации параметров продемонстрирована в листинге 3.14.

Листинг 3.14 — Метод `CheckWinCondition` в классе `PhysicsPuzzleView`

```
private void CheckWinCondition(float _)
{
    if (_sliders == null || _sliders.Length == 0) return;
```

```

        bool isWin = true;
        foreach (var slider in _sliders)
        {
            if (slider == null) continue;
            if (slider.value < _targetMin || slider.value >
_targetMax)
            {
                isWin = false;
                break;
            }
        }

        if (_applyButton != null)
        {
            _applyButton.interactable = isWin;
        }
    }
}

```

В случае если значение хотя бы одного индикатора выходит за пределы допустимого диапазона, булевой переменной `isWin` присваивается значение `false`, а кнопка подтверждения (`_applyButton`) переводится в неактивное состояние. Синхронизация всех трех параметров активирует элемент управления, позволяя пользователю передать команду на применение исправлений к целевому объекту.

Нажатие кнопки подтверждения вызывает метод `OnWin()`, который, в свою очередь, обращается к методу `FixBug()` в классе `BuggablePhysicsBox`. Данный компонент отвечает за физическое представление нестабильного куба в пространстве сцены. Программная реализация восстановления свойств объекта представлена в листинге 3.15.

Листинг 3.15 — Метод `FixBug` в классе `BuggablePhysicsBox`

```

public override void FixBug()
{
    IsBugged = false;

    _rb.isKinematic = false;
    _rb.mass = 10f;

    if (_outline)
    {
        _outline.OutlineColor = Color.green;
        Invoke(nameof(DisableOutline), 1f);
    }

    Debug.Log("Ящик починен! Теперь его можно двигать.");
}

```

Выполнение метода `FixBug` алгоритмически устраняет программную аномалию: отключается кинематический режим компонента `Rigidbody`, а показатель массы объекта нормализуется (например, снижается со значения 9999

до 10). В результате куб, подчиняясь восстановленным законам гравитации, падает на расположенную под ним нажимную плиту. Физический контакт объекта с триггером плиты замыкает цепь логического узла, подготавливая виртуальную сцену к финальному этапу прохождения уровня.

3.3.7 Завершение сценарной последовательности и подсистема подведения итогов

После активации триггерного механизма нажимной плиты восстанавливается логическая цепь, управляющая финальной преградой (дверью). Приближение к заблокированному проходу инициирует появление информационного уведомления, указывающего на то, что физическое открытие шлюза в текущей поврежденной ветке невозможно из-за конфликта геометрии или отсутствия необходимых анимационных контроллеров.

Для преодоления последнего препятствия требуется финальное применение портативного терминала и возврат в базовую (стабильную) реальность. Поскольку логическое условие открытия двери было выполнено в слое Alpha (масса куба восстановлена, плита нажата), синхронизация состояний гарантирует, что в ветке Main проход будет алгоритмически открыт. Переход в стабильную среду позволяет персонажу беспрепятственно пересечь дверной проем.

Вхождение персонажа в финальную зону активирует скрытый пространственный триггер, который сигнализирует системе об успешном завершении демонстрационного сценария. Обработка данного события делегируется менеджеру графических интерфейсов (ViewManager), который инициирует процесс затемнения экрана и выводит специализированное окно подведения итогов (EndGameView). Программная реализация данного графического компонента представлена в листинге 3.16.

Листинг 3.16 — Класс EndGameView

```
using System;
using Core.UI;
using UnityEngine;
using UnityEngine.UI;

namespace Gameplay.UI.Views.Gameplay
{
    public class EndGameView : View
    {
        [SerializeField] private Button restartButton;

        public void AddRestartButtonListener(Action callback)
        {
            restartButton.onClick.AddListener(callback.Invoke);
        }
    }
}
```

Представленный класс унаследован от базового компонента View, что позволяет централизованно управлять его отображением и скрыванием через системные вызовы. Внутри класса инкапсулирована логика подписки на элемент управления (кнопку перезапуска), что предоставляет пользователю возможность повторной инициализации игрового цикла или возврата в главное меню приложения.

На данном этапе основной цикл взаимодействия полностью замыкается. Разработанные механики сканирования, динамической смены слоев реальности, боевого взаимодействия и алгоритмической отладки физических свойств объектов формируют целостный пользовательский опыт, подтверждая жизнеспособность заложенной концепции.

3.4 Тестирование прототипа, анализ производительности и сценарии дальнейшего масштабирования

Заключительным этапом разработки вертикального среза интерактивной системы стало комплексное тестирование реализованных программных модулей и профилирование производительности вычислительных алгоритмов. Тестирование осуществлялось в интегрированной среде разработки Unity с

использованием встроенных инструментов анализа метрик (Unity Profiler) [3.16, 3.17].

Основной задачей функционального тестирования являлась проверка надежности архитектуры, основанной на менеджерах состояний (State Machine) и шине событий (EventBus). Результаты показали, что изоляция логики поведения в дискретных классах-состояниях исключила возможность возникновения логических коллизий (например, одновременного выполнения алгоритмов спринта и оглушения). Использование метода расширения `.AddTo(gameObject)` при подписке на события доказало свою эффективность: мониторинг оперативной памяти не выявил утечек при многократном создании и уничтожении агентов искусственного интеллекта.

Ключевым объектом анализа производительности стала механика динамической смены слоев реальности (Branch Hopping). Текущая архитектурная реализация, базирующаяся на вызове методов `SetActive(true/false)` для крупных иерархических структур окружения, продемонстрировала допустимые результаты в рамках прототипа. Вспышка графического интерфейса успешно маскировала визуальные артефакты перестроения геометрии.

Тем не менее, профилирование выявило кратковременные пики нагрузки (Spikes) на центральный процессор в моменты переключения. Данные пики обусловлены тем, что алгоритмы движка Unity принудительно пересчитывают иерархию физических коллайдеров и обновляют графы навигационных сеток (NavMesh) при активации ранее скрытых объектов.

Опираясь на полученные метрические данные, были разработаны следующие сценарии дальнейшего масштабирования и оптимизации программного продукта перед выпуском релизной версии:

1. Переход на систему послойного рендеринга (Culling Masks). Для обеспечения абсолютной алгоритмической бесшовности при смене веток планируется отказ от жесткого отключения геометрии. Альтернативные версии окружения будут постоянно активны и загружены в оперативную память, но размещены на изолированных физических слоях.

Переключение реальности будет осуществляться исключительно путем модификации битовой маски рендеринга виртуальной камеры, что позволит свести вычислительные затраты к выполнению одной математической операции за кадр.

2. Масштабирование паттерна Object Pool. Успешный опыт применения пула объектов в подсистеме AudioManager обосновывает необходимость распространения данного паттерна на боевую систему. Циклическая генерация снарядов (класс Projectile) и процедурная генерация выпадающих фрагментов кода (CodeBlockPickup) будут переведены на систему пулинга, что радикально снизит нагрузку на системный сборщик мусора (Garbage Collector) [3.18].

3. Асинхронная загрузка сценарных блоков. При расширении топологии виртуального мира монолитные конфигурации веток будут декомпозированы на потоковые секторы (Chunks), загружаемые в оперативную память асинхронно по мере приближения цифрового аватара.

Внедрение предложенных архитектурных модификаций позволит повысить частоту смены кадров, обеспечит плавность игрового процесса на системах со средними техническими характеристиками и подготовит программный фундамент для разработки масштабных высокодетализированных уровней [3.19, 3.20, 3.21].

3.5 Техническое обеспечение

Функционирование разработанного программного продукта требует наличия аппаратного обеспечения, соответствующего определенным критериям. В связи с использованием ресурсоемких шейдерных программ, вычисляющих смещение вершин в реальном времени для генерации визуальных искажений, а

также необходимостью одновременного нахождения в оперативной памяти данных нескольких конфигураций топологии уровня, были сформулированы технические требования к вычислительной системе.

К минимальным техническим требованиям для запуска и корректной работы приложения относятся следующие характеристики:

- операционная система: Windows 10 (64-разрядная архитектура);
- центральный процессор: Intel Core i5-4460 или аналог от AMD;
- оперативная память: 8 гигабайт;
- графический ускоритель: NVIDIA GeForce GTX 960 или аналог с поддержкой DirectX 11;
- свободное дисковое пространство: 5 гигабайт.

Для обеспечения высокой частоты кадров при инициации ресурсоемких транзакций по смене слоев виртуального пространства рекомендуются следующие параметры технического обеспечения:

- операционная система: Windows 10 или Windows 11 (64-разрядная архитектура);
- центральный процессор: Intel Core i7-8700K или аналог от AMD;
- оперативная память: 16 гигабайт;
- графический ускоритель: NVIDIA GeForce GTX 1070 или более современный аналог;
- тип накопителя: твердотельный накопитель (SSD) для минимизации задержек при асинхронной загрузке данных.

Таким образом, выбранный технологический стек в полной мере покрывает потребности проекта в инструментах для конструирования сложной архитектуры, а спецификации аппаратного обеспечения гарантируют поддержание заявленного уровня производительности при обработке динамических преобразований сцены.

Выводы

В технологическом разделе дипломной работы были детально рассмотрены процессы проектирования и программной реализации комплекса систем интерактивного приложения. Обоснован выбор игрового движка Unity, языка C# и вспомогательных библиотек. Спроектировано информационное обеспечение, гарантирующее персистентность данных посредством паттерна Repository.

Разработана и успешно протестирована децентрализованная архитектура на базе IoC-контейнера Reflex и глобальной событийной шины. В рамках программного обеспечения были реализованы инновационные механики динамической трансформации пространства, аналитического сканирования объектов и модульная боевая система, функционирующая на принципах конечного автомата. Анализ производительности подтвердил жизнеспособность созданного прототипа и позволил сформировать конкретный план архитектурной оптимизации для дальнейшего масштабирования программного продукта.

ЗАКЛЮЧЕНИЕ

поставленная цель достигнута, задачи выполнены, разработано интерактивное игровое решение с возможностью изменения сценария во время игрового процесса.

В рамках первого раздела был проведен комплексный теоретический и методологический анализ аспектов конструирования интерактивных систем с вариативным игровым процессом и динамически изменяемым виртуальным окружением.

Были исследованы существующие программные продукты, обладающие механиками пространственно-временной трансформации и элементами деконструкции коммуникативного барьера между пользователем и вычислительной системой. На основе сформированной системы узкоспециализированных критериев был произведен сравнительный анализ, результаты которого продемонстрировали отсутствие на рынке решений, в полной мере объединяющих технологии бесшовного изменения топологии уровня с мета-нарративным инструментарием симуляции программных сбоев.

Опираясь на выявленные концептуальные недостатки проанализированных проектов, были сформированы точные функциональные требования к разрабатываемому программному обеспечению. Сформированный перечень задач послужит фундаментальной теоретической базой для последующего проектирования архитектуры интерактивной системы.

Был определен и детально обоснован инструментальный аппарат, требуемый для алгоритмической реализации заявленного функционала. В качестве оптимальной среды разработки был выбран кроссплатформенный игровой движок Unity в интеграции с объектно-ориентированным языком программирования C#. Также был утвержден перечень применимых архитектурных паттернов проектирования, необходимых для обеспечения

модульности, масштабируемости и высокой отказоустойчивости программного кода разрабатываемого продукта.

В рамках второго раздела было осуществлено комплексное проектирование архитектуры интерактивной системы, базирующейся на механике динамического изменения сценария. На основе сформированных системных требований была разработана концептуальная модель программного продукта Reverse Assembly, использующая парадигму систем контроля версий в качестве фундаментальной структуры виртуального пространства.

В ходе выполнения проектных работ были достигнуты следующие результаты:

- сформирована база многослойного мира, разделенного на изолированные логические состояния (ветки Main и Alpha), что обеспечило платформу для реализации пространственной вариативности и нелинейного преодоления препятствий;
- спроектирована схема алгоритма основного игрового цикла, регламентирующая последовательность действий пользователя от этапа пространственного исследования до алгоритмической трансформации сцены и последующей стабилизации обновленной геометрии;
- детализирована логика функционирования механики переключения веток, выступающей в качестве главного инструмента глобального изменения сценария посредством переключения между альтернативными версиями уровня с учетом концепции якорных объектов;
- разработаны алгоритмы локального взаимодействия с нестабильной виртуальной средой, включающие механики устранения процедурных ошибок в статических объектах и преодоления классифицированных пространственно-временных искажений;
- спроектирован внутриигровой пользовательский интерфейс портативного терминала, интегрированный непосредственно в трехмерную сцену, и

формализована подсистема сбора данных, обеспечивающая асинхронное раскрытие сюжетной информации.

Таким образом, разработанная проектная документация, включающая формализацию логических алгоритмов, принципы архитектурного выравнивания объектов и правила балансировки вычислительных ресурсов, полностью удовлетворяет заданным спецификациям. Сформированная структурная база снижает риски возникновения программных коллизий при перестроении геометрии уровня и предоставляет исчерпывающий инструментальный аппарат для последующей технической реализации прототипа разрабатываемого решения в среде движка Unity.

В рамках третьего раздела были детально рассмотрены процессы проектирования и программной реализации комплекса систем интерактивного приложения. Обоснован выбор игрового движка Unity, языка C# и вспомогательных библиотек. Спроектировано информационное обеспечение, гарантирующее персистентность данных посредством паттерна Repository.

Разработана и успешно протестирована децентрализованная архитектура на базе IoC-контейнера Reflex и глобальной событийной шины. В рамках программного обеспечения были реализованы инновационные механики динамической трансформации пространства, аналитического сканирования объектов и модульная боевая система, функционирующая на принципах конечного автомата. Анализ производительности подтвердил жизнеспособность созданного прототипа и позволил сформировать конкретный план архитектурной оптимизации для дальнейшего масштабирования программного продукта.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. АНАЛИТИЧЕСКИЙ РАЗДЕЛ

- 1.1.GDC. State of the Game Industry 2024 Report. — 2024. — URL: <https://reg.gdconf.com/state-of-game-industry-2024> (дата обращения: 20.05.2026) — Текст электронный.
- 1.2.Unity. 2024 Unity Gaming Report Highlights Game Studios' Continued Resilience As They Boldly Stretch Resources Amidst Shifting Market Forces. — 2024. — URL: <https://investors.unity.com/news/news-details/2024/2024-Unity-Gaming-Report-Highlights-Game-Studios-Continued-Resilience-As-They-Boldly-Stretch-Resources-Amidst-Shifting-Market-Forces/default.aspx> (дата обращения: 20.05.2026) — Текст электронный.
- 1.3.Newzoo. 2024 Global Games Market Report. — 2024. — URL: https://best-of-gaming.be/wp-content/uploads/2024/09/2024_Newzoo_Global_Games_Market_Report.pdf (дата обращения: 20.05.2026) — Текст электронный.
- 1.4.Unity. 2025 Unity Gaming Report: Gaming Industry Trends. — 2025. — URL: <https://unity.com/resources/gaming-report-2025> (дата обращения: 20.05.2026) — Текст электронный.
- 1.5.Game Developer. A close analysis of Titanfall 2's level Effect and Cause. — 2023. — URL: <https://www.gamedeveloper.com/design/a-close-analysis-of-i-titanfall-2-s-i-level-effect-and-cause> (дата обращения: 20.05.2026) — Текст электронный.
- 1.6.Adams S. Re-Calibrating Steampunk London: Heterotopia and Spatial Imaginaries in Assassin's Creed: Syndicate and The Order 1886 // Humanities. — 2021. — DOI: 10.3390/H10010056. — URL: <https://www.mdpi.com/2076-0787/10/1/56> (дата обращения: 20.05.2026) — Текст электронный.
- 1.7.Materiality, Nonlinearity, and Interpretive Openness in Contemporary Archaeogames // Eludamos. — 2022. — DOI: 10.7557/23.6618. —

- URL: <https://septentrio.uit.no/index.php/eludamos/article/view/6618> (дата обращения: 20.05.2026) — Текст электронный.
- 1.8. The Stanley Parable: Ultra Deluxe — A Metanarrative Masterpiece. — 2022. — URL: (дата обращения: 20.05.2026) — Текст электронный.
- 1.9. Magerko B., et al. Educating Interactive Narrative Designers: Cornerstones of a Program // Transactions of the Digital Games Research Association. — 2021. — DOI: 10.26503/todigra.v5i3.125. — URL: <http://todigra.org/index.php/todigra/article/view/1794> (дата обращения: 20.05.2026) — Текст электронный.
- 1.10. Veloso C., Prada R. Validating the plot of Interactive Narrative games // 2021 IEEE Conference on Games. — 2021. — DOI: 10.1109/CoG52621.2021.9618897. — URL: <https://ieeexplore.ieee.org/document/9618897/> (дата обращения: 20.05.2026) — Текст электронный.
- 1.11. Serious Interactive Digital Narrative: Explorations in Personalization and Player Experience Enrichment // IEEE. — 2021. — DOI: 10.1109/ISGS54702.2021.9685015. — URL: <https://ieeexplore.ieee.org/document/9685015/> (дата обращения: 20.05.2026) — Текст электронный.
- 1.12. Unity. How to immerse your players through effective UI and game design // Unity Blog. — 2022. — URL: <https://unity.com/blog/games/how-to-immerse-your-players-through-effective-ui-and-game-design> (дата обращения: 20.05.2026) — Текст электронный.
- 1.13. Wiratama C., Wiguna A., Kurniawan A. Can Diegetic User Interface Improves Immersion in Role-Playing Games? // 2021 International Conference on Electrical Engineering and Informatics. — 2021. — DOI: 10.1109/ICEEIE52663.2021.9616732. — URL: <https://ieeexplore.ieee.org/document/9616732/> (дата обращения: 20.05.2026) — Текст электронный.

- 1.14. Andersson R. Effects of Diegetic User Interface on Immersion and User Experience. — 2024. — URL: https://www.theseus.fi/bitstream/handle/10024/862563/Andersson_Roni.pdf?sequence=2&isAllowed=y (дата обращения: 20.05.2026) — Текст электронный.
- 1.15. Caroux L., Isbister K., Le Bigot L., Vibert N. Video Game Interfaces and Diegesis: The Impact on Experts and Novices' Performance and Experience in Virtual Reality // International Journal of Human-Computer Interaction. — 2021. — DOI: 10.1080/10447318.2020.1870819. — URL: <https://www.tandfonline.com/doi/full/10.1080/10447318.2020.1870819> (дата обращения: 20.05.2026) — Текст электронный.
- 1.16. Unity. Systems that create ecosystems: Emergent game design // Unity Blog. — 2021. — URL: <https://unity.com/blog/games/systems-that-create-ecosystems-emergent-game-design> (дата обращения: 20.05.2026) — Текст электронный.
- 1.17. Exploring emergent co-creative narrative in a strategy video game brand // Journal of Aesthetics & Culture. — 2021. — DOI: 10.1080/16522354.2021.1959705. — URL: <https://www.tandfonline.com/doi/full/10.1080/16522354.2021.1959705> (дата обращения: 20.05.2026) — Текст электронный.
- 1.18. PANGeA: Procedural Artificial Narrative Using Generative AI for Turn-Based, Role-Playing Video Games // AAAI AIIDE. — 2024. — DOI: 10.1609/aiide.v20i1.31876. — URL: <https://ojs.aaai.org/index.php/AIIDE/article/view/31876> (дата обращения: 20.05.2026) — Текст электронный.
- 1.19. Large Language Models and Video Games: A Preliminary Scoping Review // ACM Digital Library. — 2024. — DOI: 10.1145/3640794.3665582. — URL: <https://dl.acm.org/doi/10.1145/3640794.3665582> (дата обращения: 20.05.2026) — Текст электронный.

- 1.20. Video games and attitude change: A meta-analysis // ScienceDirect. — 2023. — URL: <https://www.sciencedirect.com/science/article/pii/S0361476X23000796> (дата обращения: 20.05.2026) — Текст электронный.
- 1.21. Player Agency in Audience Gaming // De Gruyter. — 2021. — DOI: 10.1515/9783839454213-007. — URL: <https://www.degruyter.com/document/doi/10.1515/9783839454213-007/html> (дата обращения: 20.05.2026) — Текст электронный.
- 1.22. Bopp J. A., Mekler E. D., Opwis K. “Naked and on Fire”: Examining Player Agency Experiences in Narrative-Focused Gameplay // CHI Conference on Human Factors in Computing Systems. — 2021. — DOI: 10.1145/3411764.3445540. — URL: <https://dl.acm.org/doi/10.1145/3411764.3445540> (дата обращения: 20.05.2026) — Текст электронный.
- 1.23. Keeping the Player on an Emotional Trajectory in Interactive Storytelling // AAAI AIIDE. — 2021. — DOI: 10.1609/aiide.v11i1.12783. — URL: <https://ojs.aaai.org/index.php/AIIDE/article/view/12783> (дата обращения: 20.05.2026) — Текст электронный.
- 1.24. Flow Experience Detection and Analysis for Game Users by Wearable-Devices-Based Physiological Responses Capture // IEEE Internet of Things Journal. — 2021. — DOI: 10.1109/IJOT.2020.3010853. — URL: <https://ieeexplore.ieee.org/document/9145561/> (дата обращения: 20.05.2026) — Текст электронный.

2. СПЕЦИАЛЬНЫЙ РАЗДЕЛ

- 2.1. GitHub. Git Guides. — 2024. — URL: <https://github.com/git-guides> (дата обращения: 20.05.2026) — Текст электронный.
- 2.2. Gridly. Version control in game development: The complete guide. — 2026. — URL: <https://www.gridly.com/blog/version-control-in-game-development/> (дата обращения: 20.05.2026) — Текст электронный.

- 2.3.Can Playfulness Be Designed? Understanding Playful Design through Agency in Astroneer (2019) // Eludamos. — 2021. — DOI: 10.7557/23.6362. — URL: <https://septentrio.uit.no/index.php/eludamos/article/view/vol12no1-3> (дата обращения: 20.05.2026) — Текст электронный.
- 2.4.A Systematic Literature Review on the User Experience Design for Game-Based Interventions via 3D Virtual Worlds in K-12 Education // Multimodal Technologies and Interaction. — 2021. — DOI: 10.3390/MTI5060028. — URL: <https://www.mdpi.com/2414-4088/5/6/28> (дата обращения: 20.05.2026) — Текст электронный.
- 2.5.Games and data capture culture: play in the era of accelerated neoliberalism // Media, Culture & Society. — 2021. — DOI: 10.1177/01634437211045556. — URL: <https://journals.sagepub.com/doi/10.1177/01634437211045556> (дата обращения: 20.05.2026) — Текст электронный.
- 2.6.Enhancing In-game Immersion Using BCI-controlled Mechanics // ACM Digital Library. — 2021. — DOI: 10.1145/3489849.3489862. — URL: <https://dl.acm.org/doi/10.1145/3489849.3489862> (дата обращения: 20.05.2026) — Текст электронный.
- 2.7.Diegetic vs. Non-Diegetic GUIs: What do Virtual Reality Players Prefer? — 2021. — DOI: 10.2312/EGVE.20211333. — URL: <https://diglib.eg.org/handle/10.2312/egve20211333> (дата обращения: 20.05.2026) — Текст электронный.
- 2.8.Brown J., Cairns P. Diegetic and Non-diegetic Health Interfaces in VR Shooter Games // Springer. — 2021. — DOI: 10.1007/978-3-030-85613-7_1. — URL: https://link.springer.com/10.1007/978-3-030-85613-7_1 (дата обращения: 20.05.2026) — Текст электронный.
- 2.9.Apple Developer. Add accessibility to your Unity games // WWDC22. — 2022. — URL: <https://developer.apple.com/videos/play/wwdc2022/10151/> (дата обращения: 20.05.2026) — Текст электронный.

- 2.10. Microsoft Game Dev. Xbox Accessibility Guidelines. — 2023. —
URL: <https://learn.microsoft.com/en-us/gaming/accessibility/guidelines> (дата обращения: 20.05.2026) — Текст электронный.
- 2.11. Xbox Wire. The Future of Game Accessibility on Xbox. — 2021. —
URL: <https://news.xbox.com/en-us/2021/02/16/xbox-accessibility-guidelines-and-testing/> (дата обращения: 20.05.2026) — Текст электронный.

3. ТЕХНОЛОГИЧЕСКИЙ РАЗДЕЛ

- 3.1.Unity. Level up your code with design patterns and SOLID E-book. — 2024.
— URL: <https://unity.com/resources/design-patterns-solid-ebook> (дата обращения: 20.05.2026) — Текст электронный.
- 3.2.Unity. Level up your code with game programming patterns. — 2024. —
URL: <https://unity.com/resources/level-up-your-code-with-game-programming-patterns> (дата обращения: 20.05.2026) — Текст электронный.
- 3.3.Baron D. Learning Design Patterns with Unity. — Packt Publishing, 2024. —
URL: <https://www.packtpub.com/en-us/product/learning-design-patterns-with-unity-9781805120285?type=print> (дата обращения: 20.05.2026) — Текст электронный.
- 3.4.Microsoft Learn. Object-Oriented programming (C#). — 2025. —
URL: <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/tutorials/oop> (дата обращения: 20.05.2026) — Текст электронный.
- 3.5.Microsoft Learn. C# language reference. — 2024. —
URL: <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/> (дата обращения: 20.05.2026) — Текст электронный.
- 3.6.Microsoft Learn. Migrate from Newtonsoft.Json to System.Text.Json. — 2025.
— URL: <https://learn.microsoft.com/en-us/dotnet/standard/serialization/system-text-json/migrate-from-newtonsoft> (дата обращения: 20.05.2026) — Текст электронный.

- 3.7. Newtonsoft. Json.NET Introduction. — 2024. —
URL: <https://www.newtonsoft.com/json/help/html/introduction.htm> (дата обращения: 20.05.2026) — Текст электронный.
- 3.8. Unity. Persistent data: How to save your game states and settings // Unity Blog. — 2021. — URL: <https://unity.com/blog/games/persistent-data-how-to-save-your-game-states-and-settings> (дата обращения: 20.05.2026) — Текст электронный.
- 3.9. Unity. Dependency injection: Cloud Code modules. — 2025. —
URL: <https://docs.unity.com/en-us/cloud-code/modules/how-to-guides/initialize-modules/dependency-injection> (дата обращения: 20.05.2026) — Текст электронный.
- 3.10. Shaggy Dev. An introduction to state machines and the state pattern. — 2021. — URL: <https://shaggydev.com/2021/11/01/state-machines-intro/> (дата обращения: 20.05.2026) — Текст электронный.
- 3.11. Unity. Addressables.LoadAsset(s)Async // Unity Manual. — 2023. —
URL: <https://docs.unity3d.com/Packages/com.unity.addressables@1.15/manual/LoadingAddressableAssets.html> (дата обращения: 20.05.2026) — Текст электронный.
- 3.12. Unity. Addressables // Unity Manual. — 2024. —
URL: <https://docs.unity3d.com/2023.2/Documentation/Manual/com.unity.addressables.html> (дата обращения: 20.05.2026) — Текст электронный.
- 3.13. Unity. Create modular game architecture in Unity with ScriptableObjects. — 2023. — URL: <https://unity.com/resources/create-modular-game-architecture-with-scriptable-objects-ebook> (дата обращения: 20.05.2026) — Текст электронный.
- 3.14. Unity. Visual Effect Graph 12.0.0 Manual. — 2023. —
URL: <https://docs.unity3d.com/Packages/com.unity.visualeffectgraph@12.0/> (дата обращения: 20.05.2026) — Текст электронный.

- 3.15. Unity Discussions. VFX graph SIGGRAPH 2021 video. — 2021. — URL: <https://discussions.unity.com/t/vfx-graph-siggraph-2021-video/862147> (дата обращения: 20.05.2026) — Текст электронный.
- 3.16. Unity. Customizing performance metrics in the Unity Profiler // Unity Blog. — 2022. — URL: <https://unity.com/blog/engine-platform/customizing-performance-metrics-in-the-unity-profiler> (дата обращения: 20.05.2026) — Текст электронный.
- 3.17. Unity. Unity 6 Optimization Guides for console, PC, mobile, web and XR // Unity Blog. — 2024. — URL: <https://unity.com/blog/unity-6-game-optimization-guides> (дата обращения: 20.05.2026) — Текст электронный.
- 3.18. Embrace. Avoiding garbage collector performance spikes in Unity. — 2023. — URL: <https://embrace.io/blog/garbage-collector-spikes-unity/> (дата обращения: 20.05.2026) — Текст электронный.
- 3.19. Unity. How to run automated tests for your games with the Unity Test Framework. — 2024. — URL: <https://unity.com/how-to/automated-tests-unity-test-framework> (дата обращения: 20.05.2026) — Текст электронный.
- 3.20. Unity Discussions. Profiling in Unity 2022 LTS: Resources to help you get started. — 2024. — URL: <https://discussions.unity.com/t/profiling-in-unity-2022-lts-resources-to-help-you-get-started/355179> (дата обращения: 20.05.2026) — Текст электронный.
- 3.21. Unity Learn. Who is a Quality Assurance Tester? — 2025. — URL: <https://learn.unity.com/course/qa-pathway/unit/introduction-to-quality-assurance-testing/tutorial/who-is-a-quality-assurance-tester?version=2018.4> (дата обращения: 20.05.2026) — Текст электронный.